



fetel@HCMUS

KHOA ĐIỆN TỬ - VIỄN THÔNG

ĐỒ ÁN THỰC HÀNH PHƯƠNG PHÁP TÍNH HK1 - NĂM HỌC: 2025-2026

Tên đồ án: Sử dụng Matlab thực hiện giao diện giải hàm và dữ liệu bằng phương pháp tính

Danh sách thành viên nhóm:	
Phạm Phú Thành	23207019
Huỳnh Đức Trọng	23207022

TP.HCM – 12/2025

—*— Lời mở đầu và sự cảm ơn —*—

XÂY DỰNG HỆ THỐNG TÍNH TOÁN TƯƠNG TÁC TRÊN GIAO DIỆN NỀN TẢNG MATLAB

Kỷ nguyên số phát triển vượt bậc, việc tối ưu hóa các mô hình toán học thông qua sức mạnh tính toán là yêu cầu tất yếu đối với sinh viên kỹ thuật. Đồ án kết thúc môn học **"Thực hành Phương pháp tính"** của nhóm chúng em là kết quả của một quá trình nghiên cứu và thực nghiệm bền bỉ. Lời đầu tiên, chúng em xin bày tỏ lòng biết ơn chân thành nhất tới **giảng viên Huỳnh Quốc Thịnh**, người đã tận tình truyền đạt kiến thức và dẫn dắt chúng em tiếp cận môn học một cách khoa học và đầy cảm hứng.

Dự án được xây dựng dựa trên nền tảng MATLAB, tận dụng tối ưu khả năng tính toán ma trận và đồ họa mạnh mẽ để số hóa các thuật toán phức tạp. Trải qua lộ trình 11 tuần học tập với 9 bài lab chuyên sâu, từ những dòng lệnh cơ bản đến việc xây dựng giao diện tương tác (GUI) phức tạp, chúng em đã hệ thống hóa kiến thức thành 6 module hoàn chỉnh: Tìm nghiệm, Nội suy, Hồi quy, Đạo hàm, Tích phân và phần giới thiệu đội ngũ thực hiện.

Đồ án này không chỉ là một sản phẩm phần mềm, mà là minh chứng cho sự chuyển đổi từ lý thuyết hàn lâm sang các giải pháp thực tiễn sinh động. Chính nhờ sự định hướng của Thầy qua từng bài thực hành, chúng em đã có cơ hội rèn luyện tư duy logic và nhận ra giá trị hiện thực to lớn của toán học trong đời sống kỹ thuật.

Mục lục

Phần 1: Giao diện Tìm nghiệm	5
1/ Code:	5
2/ Kết quả giao diện:.....	8
3/ Minh họa thuật toán:.....	9
4/ Minh họa kết quả:.....	11
- Phương pháp chia đôi:.....	11
- Phương pháp lặp:.....	11
- Phương pháp Newton(tiếp tuyến):	12
5/ Mô tả hoạt động:.....	12
Phần 2: Giao diện Nội suy	14
1/ Code:	14
2/ Kết quả giao diện:.....	16
3/ Minh họa thuật toán:.....	16
4/ Minh họa kết quả:.....	18
- Phương pháp Lagrange:.....	18
- Phương pháp Newton:	18
5/ Mô tả hoạt động:.....	19
Phần 3: Giao diện Hồi quy	21
1/ Code:	21
2/ Kết quả giao diện:.....	24
3/ Minh họa thuật toán:.....	25
4/ Minh họa kết quả:.....	27
- Phương pháp tuyến tính:.....	27
- Phương pháp hàm mũ:.....	28
- Phương pháp Logarit:	28
5/ Mô tả hoạt động:.....	29

Phần 4: Giao diện Đạo hàm	31
1/ Code:	31
2/ Kết quả giao diện:.....	33
3/ Minh họa thuật toán:.....	33
4/ Minh họa kết quả:.....	35
Giá trị sai $O(h)$:	35
Giá trị sai $O(h^2)$:.....	38
5/ Mô tả hoạt động:.....	41
Phần 5: Giao diện tích phân	43
1/ Code:	43
2/ Kết quả giao diện:.....	45
3/ Minh họa thuật toán:.....	46
4/ Minh họa kết quả:.....	46
Phương pháp hình thang:	47
Phương pháp simpson 1/3:	47
Phương pháp simpson 3/8:	48
5/ Mô tả hoạt động:.....	49
Phần 6: Giới thiệu nhóm	49
1/ Code:	51
2/ Kết quả giao diện:.....	51
3/ Minh họa thuật toán:.....	52
4/ Minh họa kết quả:.....	52
5/ Mô tả hoạt động:.....	52
TỔNG KẾT VÀ ĐÁNH GIÁ:	53
A. Tiêu chí đánh giá ứng dụng: (70%).....	53
B. Thang điểm đánh giá báo cáo word: (30%).....	54

PHẦN 1: GIAO DIỆN TÌM NGHIỆM

1/ Code:

```
% Button pushed function: RunButton
function RunButtonPushed(app, event)

% Tìm nghiệm

% 1. THU THẬP DỮ LIỆU ĐẦU VÀO

f_str = app.NhphngtrnhEditField.Value; % Lấy phương trình dưới dạng chuỗi
khoang_str = app.NhpkhongphnlynghimEditField.Value; % Lấy khoảng phân ly nghiệm [a, b]

% Chuyển chuỗi khoảng phân ly thành 2 giá trị số [a, b]
try
    % Nhập chuỗi bằng dấu phẩy, nếu không được thì cách nhau bằng khoảng trắng
    khoang_parts = strsplit(khoang_str, ',');
    if length(khoang_parts) ~= 2
        khoang_parts = strsplit(khoang_str, ' ');
    end

    a = str2double(strtrim(khoang_parts{1}));
    b = str2double(strtrim(khoang_parts{2}));
    if isnan(a) || isnan(b)
        uialert(app.UIFigure, 'Vui lòng nhập khoảng [a, b] đúng định dạng (ví dụ: 1, 2)', 'Lỗi đầu vào');
        return;
    end
catch
    uialert(app.UIFigure, 'Vui lòng nhập khoảng [a, b] đúng định dạng (ví dụ: 1, 2)', 'Lỗi đầu vào');
    return;
end

% Lấy sai số cho phép
epsilon = app.NhpsaischophpEditField.Value;

% Lấy phương pháp đã chọn
```

```
% Lấy phương pháp đã chọn
phuong_phap = app.ChnphngphptmDropDown.Value;

% Khởi tạo hàm f(x)
f = str2sym(f_str);
f_func = matlabFunction(f);

% Khởi tạo biến kết quả
x_nghiem = NaN;
so_lan_lap = 0;

% 2. VẼ ĐỒ THỊ TRƯỚC KHI TÌM NGHIỆM

try
    % Tạo khoảng X để vẽ đồ thị
    plot_min = min(a, b) - 0.5;
    plot_max = max(a, b) + 0.5;
    X = linspace(plot_min, plot_max, 500);
    Y = f_func(X);

    % Vẽ đồ thị
    plot(app.UIAxes, X, Y, 'LineWidth', 2);
    hold(app.UIAxes, 'on');
    plot(app.UIAxes, [plot_min plot_max], [0 0], 'r--');

    % Đánh dấu khoảng [a, b]
    plot(app.UIAxes, [a a], ylim(app.UIAxes), 'g:');
    plot(app.UIAxes, [b b], ylim(app.UIAxes), 'g:');

    title(app.UIAxes, 'Đồ thị hàm số và Khoảng phân ly');
    xlabel(app.UIAxes, 'X');
```

```

        title(app.UIAxes, 'Đồ thị hàm số và Khoảng phân ly');
        xlabel(app.UIAxes, 'X');
        ylabel(app.UIAxes, 'Y');
        hold(app.UIAxes, 'off');

    catch ME
        uialert(app.UIFigure, ['Lỗi khi vẽ đồ thị: ' ME.message], 'Lỗi Đồ thị');
        return;
    end

    % 3. THỰC HIỆN TÍNH TOÁN NGHIỆM SỐ
    if strcmp(phuong_phap, 'Chia đôi') % PHƯƠNG PHÁP CHIA ĐÔI

        % Kiểm tra điều kiện phân ly nghiệm
        if f_func(a) * f_func(b) > 0
            uialert(app.UIFigure, 'f(a) và f(b) cùng dấu. Vui lòng chọn khoảng phân ly nghiệm khác.', 'Lỗi Thuật toán');
            return;
        end

        c = (a + b) / 2;
        so_lan_lap = 0;

        while (b - a) / 2 > epsilon
            so_lan_lap = so_lan_lap + 1;
            c = (a + b) / 2;

            if f_func(c) == 0
                break;
            elseif f_func(a) * f_func(c) < 0
                b = c;
            end
        end
    end

```

```

        if f_func(c) == 0
            break;
        elseif f_func(a) * f_func(c) < 0
            b = c;
        else
            a = c;
        end
    end
    x_nghiem = (a + b) / 2;

    elseif strcmp(phuong_phap, 'Lập') % PHƯƠNG PHÁP LẬP

        % Yêu cầu nhập giá trị khởi tạo x0
        x0 = (a + b) / 2;

        % Lấy đạo hàm của f(x)
        syms x
        f_sym = str2sym(f_str);

        % Hàm lập đơn giản: x = g(x). Ta chọn g(x) sao cho |g'(x)| < 1 trên [a, b].
        df_sym = diff(f_sym, x);
        df_func = matlabFunction(df_sym);

        % Chọn c = 1/max|f'(x)| trên [a, b].
        if abs(df_func(x0)) < 1e-6
            uialert(app.UIFigure, 'Đạo hàm tại điểm khởi tạo quá nhỏ. Thử phương pháp khác hoặc khoảng khác.', 'Lỗi Thuật toán');
            return;
        end

        c = 1 / df_func(x0);
        g_sym = x - c * f_sym;
        g_func = matlabFunction(g_sym);
    end

```

```

% Kiểm tra điều kiện lặp:  $|g'(x)| < 1$ 
dg_sym = diff(g_sym, x);
dg_func = matlabFunction(dg_sym);

% Kiểm tra  $|g'(x)|$  tại 2 cận
if abs(dg_func(a)) >= 1 || abs(dg_func(b)) >= 1
    uialert(app.UIFigure, 'Điều kiện hội tụ  $|g''(x)| < 1$  không đảm bảo trên khoảng này. Thử khoảng khác.', 'Lỗi Hội tụ');
    return;
end

x_prev = x0;
so_lan_lap = 0;

for k = 1:100 % Giới hạn số lần lặp
    x_curr = g_func(x_prev);
    so_lan_lap = k;

    if abs(x_curr - x_prev) < epsilon
        x_nghiem = x_curr;
        break;
    end
    x_prev = x_curr;
end

elseif strcmp(phuong_phap, 'Newton (tiếp tuyến)') % PHƯƠNG PHÁP NEWTON

% Yêu cầu nhập giá trị khởi tạo x0
x0 = (a + b) / 2;

% Lấy đạo hàm của f(x)
syms x
f_sym = str2sym(f_str);
df_sym = diff(f_sym, x);

```

```

% Lấy đạo hàm của f(x)
syms x
f_sym = str2sym(f_str);
df_sym = diff(f_sym, x);

f_func = matlabFunction(f_sym);
df_func = matlabFunction(df_sym);

x_prev = x0;
so_lan_lap = 0;

for k = 1:100 % Giới hạn số lần lặp
    so_lan_lap = k;

    % Kiểm tra đạo hàm f'(x) khác 0
    df_val = df_func(x_prev);
    if abs(df_val) < 1e-8
        uialert(app.UIFigure, 'Đạo hàm f'(x) gần bằng 0. Thuật toán không hội tụ.', 'Lỗi Thuật toán');
        return;
    end

    x_curr = x_prev - f_func(x_prev) / df_val;

    if abs(x_curr - x_prev) < epsilon
        x_nghiem = x_curr;
        break;
    end
    x_prev = x_curr;
end

end

```

% 4. HIỂN THỊ KẾT QUẢ VÀ ĐÁNH DẤU TRÊN ĐỒ THỊ

```
if ~isnan(x_nghiem)

    % Hiển thị nghiệm (đã hội tụ)
    app.KtqunghimEditField.Value = x_nghiem;
    app.SlnlpEditField.Value = so_lan_lap;

    % Đánh dấu nghiệm trên đồ thị
    hold(app.UIAxes, 'on');
    plot(app.UIAxes, x_nghiem, f_func(x_nghiem), 'go', 'MarkerSize', 8, 'LineWidth', 2);
    hold(app.UIAxes, 'off');

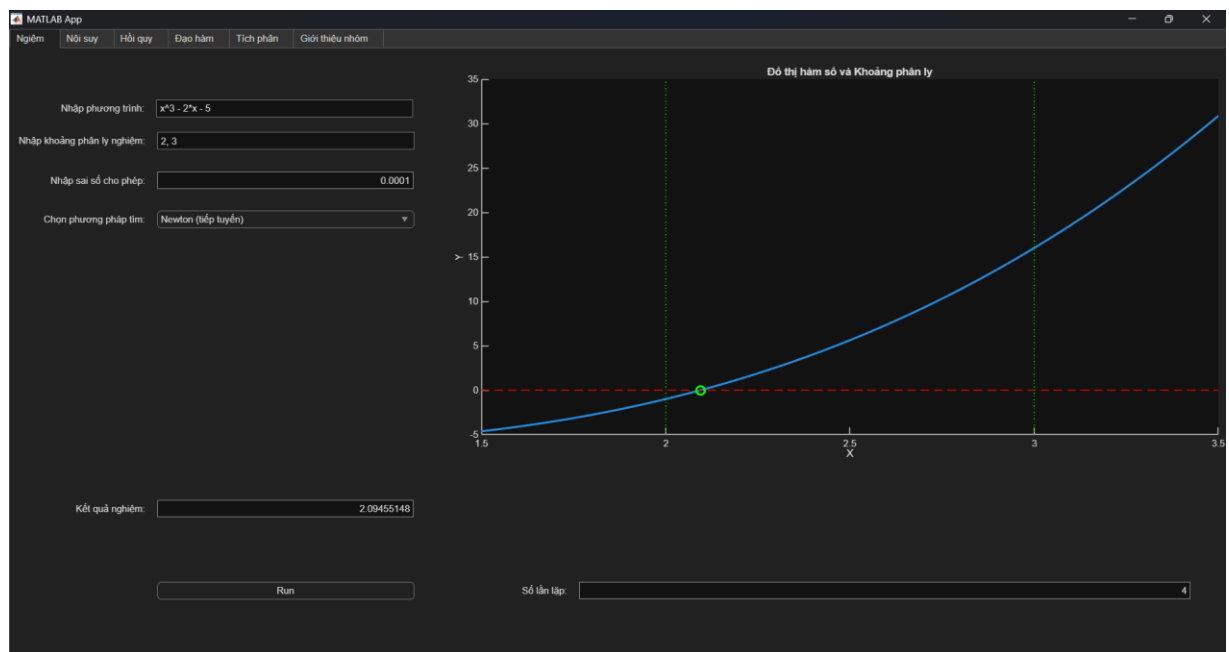
else

    % Hiển thị nếu không hội tụ
    app.KtqunghimEditField.Value = NaN;
    app.SlnlpEditField.Value = so_lan_lap;
    uialert(app.UIFigure, 'Không tìm thấy nghiệm hội tụ sau 100 lần lặp.', 'Lỗi Hội tụ');

end

end
```

2/ Kết quả giao diện:



3/ Minh họa thuật toán:

1. Minh họa thuật toán tìm nghiệm (tổng quát)

Bước 1. Nhập:

- Hàm số $f(x)$
- Khoảng phân ly nghiệm $[a, b]$
- Sai số cho phép ε
- Phương pháp tìm nghiệm

Bước 2. Kiểm tra điều kiện áp dụng của từng phương pháp.

Bước 3. Thực hiện thuật toán lặp tương ứng.

Bước 4. Dừng khi điều kiện sai số thỏa:

$$|x_{k+1} - x_k| \leq \varepsilon$$

2. Minh họa thuật toán Chia đôi

Ý tưởng: Thu hẹp dần khoảng chứa nghiệm.

Các bước:

1. Chọn khoảng $[a, b]$ sao cho

$$f(a) \cdot f(b) < 0$$

2. Tính trung điểm:

$$c = \frac{a + b}{2}$$

3. Nếu $f(c) = 0 \rightarrow$ nghiệm chính xác.
4. Nếu $f(a) \cdot f(c) < 0 \rightarrow$ đặt $b = c$, ngược lại $a = c$.
5. Lặp lại cho đến khi:

$$\frac{b - a}{2} \leq \varepsilon$$

6. Nghiệm xấp xỉ:

$$x \approx \frac{a + b}{2}$$

3. Minh họa thuật toán Lặp đơn

Ý tưởng: Biến phương trình $f(x) = 0$ thành $x = g(x)$.

Các bước:

1. Chọn hàm lặp $x = g(x)$.
2. Chọn giá trị ban đầu x_0 .
3. Kiểm tra điều kiện hội tụ:

$$|g'(x)| < 1 \text{ trên } [a, b]$$

4. Lặp:

$$x_{k+1} = g(x_k)$$

5. Dừng khi:

$$|x_{k+1} - x_k| \leq \varepsilon$$

4. Minh họa thuật toán Newton (tiếp tuyến)

Ý tưởng: Dùng tiếp tuyến tại điểm gần nghiệm.

Các bước:

1. Chọn điểm khởi tạo x_0 .
2. Tính đạo hàm $f'(x)$.
3. Lặp theo công thức:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

4. Kiểm tra điều kiện:

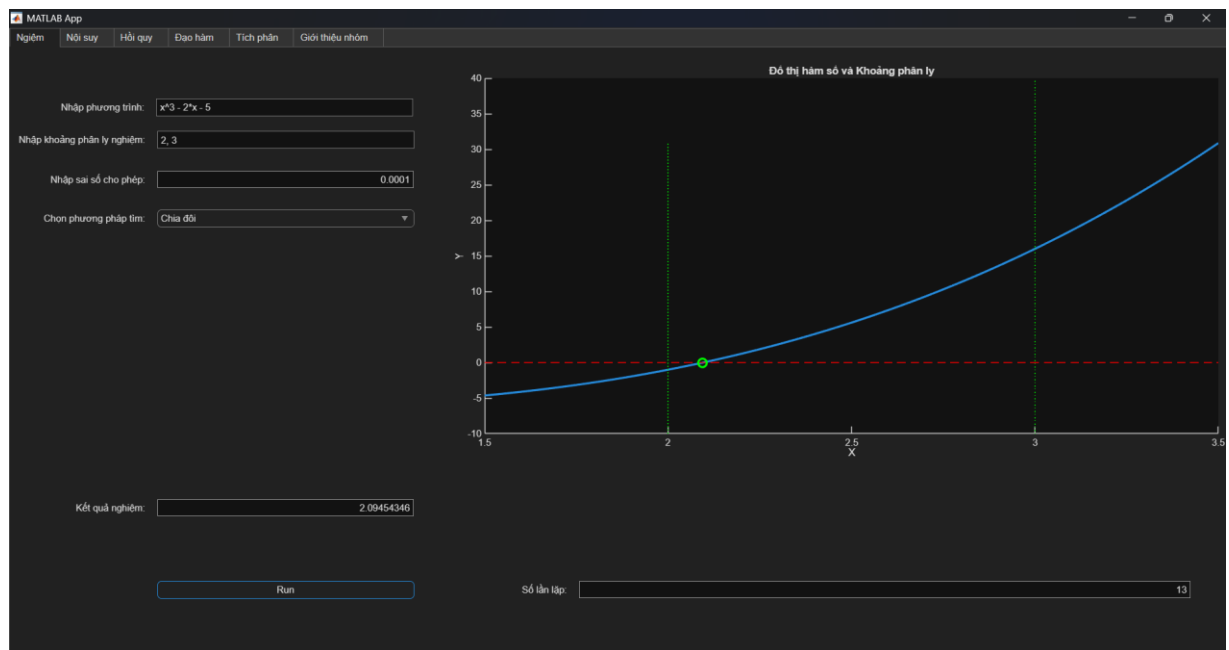
$$f'(x_k) \neq 0$$

5. Dừng khi:

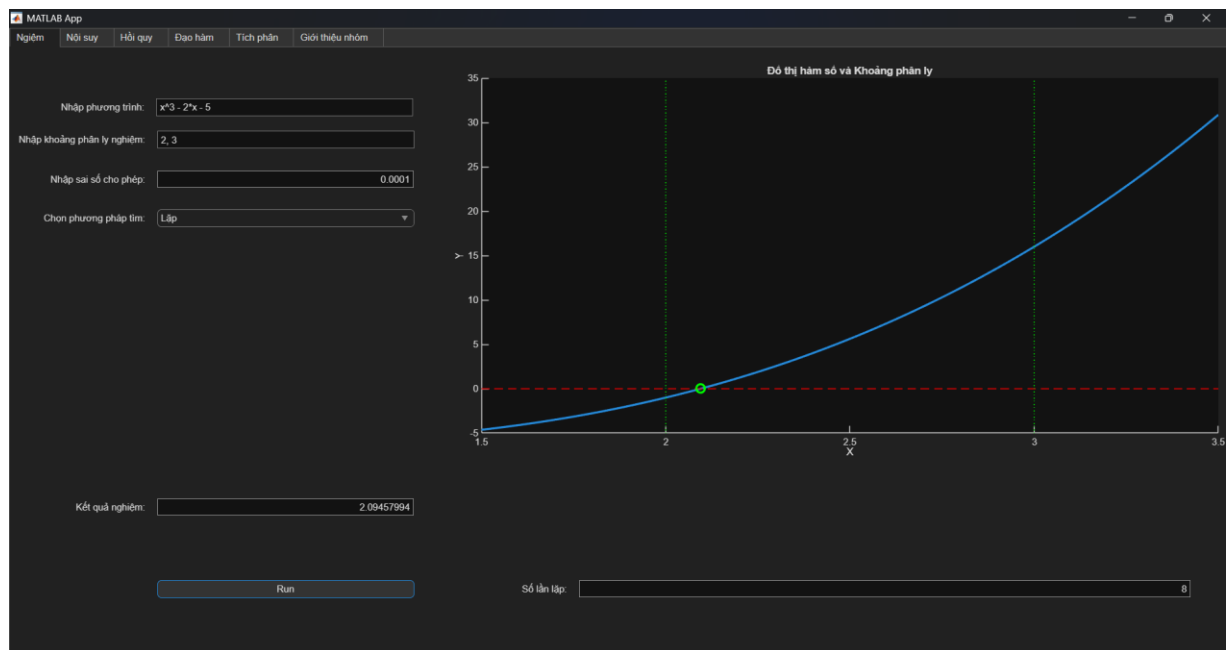
$$|x_{k+1} - x_k| \leq \varepsilon$$

4/ Minh họa kết quả:

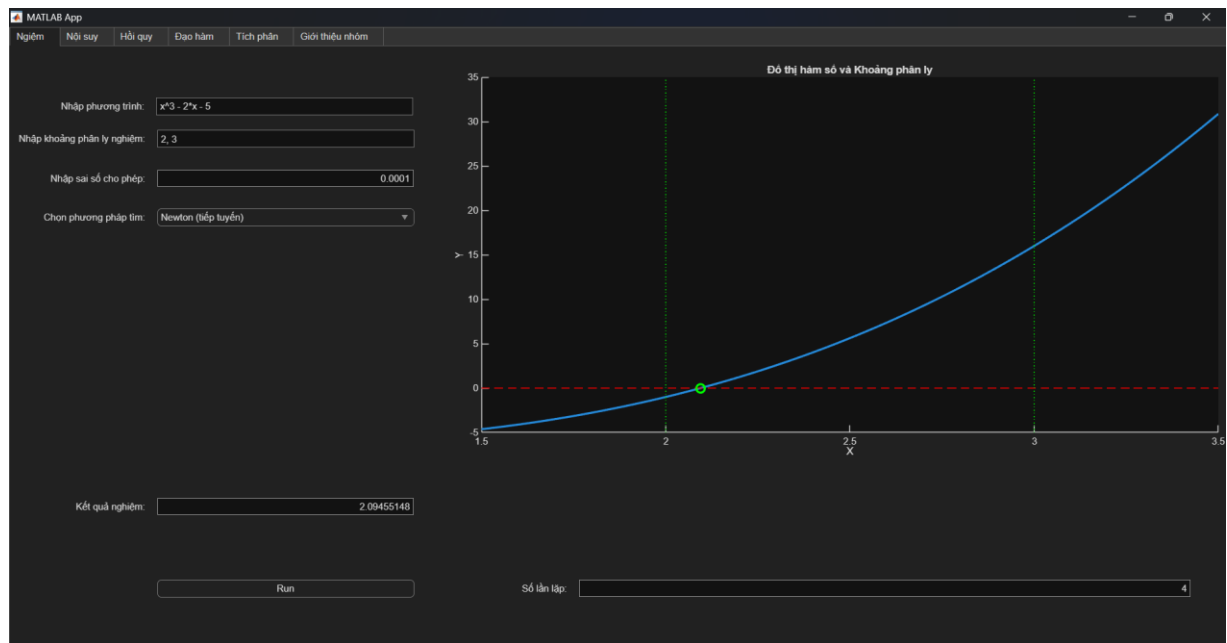
- Phương pháp chia đôi:



- Phương pháp lặp:



- Phương pháp Newton(tiếp tuyến):



5/ Mô tả hoạt động:

Mã nguồn là một hàm xử lý sự kiện (callback) cho nút **Run** trong một ứng dụng MATLAB App Designer. Ứng dụng này dùng để giải phương trình phi tuyến $f(x) = 0$ bằng các phương pháp số học.

Dưới đây là mô tả chi tiết hoạt động của chương trình theo từng bước:

1. Thu thập và xử lý dữ liệu đầu vào

Chương trình bắt đầu bằng việc đọc dữ liệu từ giao diện người dùng (UI):

- **Phương trình:** Lấy chuỗi ký tự (vd: $x^3 - 2*x - 5$) và chuyển thành hàm số trong MATLAB bằng `str2sym` và `matlabFunction`.
- **Khoảng phân ly [a, b]:** Tách chuỗi (bằng dấu phẩy hoặc khoảng trắng) để lấy hai điểm đầu mút. Đây là vùng mà người dùng nghi ngờ có nghiệm.
- **Sai số (ϵ):** Độ chính xác mong muốn cho nghiệm.
- **Phương pháp:** Người dùng chọn một trong ba phương pháp: **Chia đôi**, **Lập đơn**, hoặc **Newton**.

2. Vẽ đồ thị hàm số

Trước khi tính toán, ứng dụng thực hiện vẽ đồ thị để người dùng trực quan hóa bài toán:

- Vẽ đường cong của hàm số $f(x)$ trong khoảng $[a - 0.5, b + 0.5]$.
 - Vẽ đường thẳng $y = 0$ (trục hoành) màu đỏ đứt đoạn để dễ thấy điểm cắt (nghiệm).
 - Đánh dấu hai biên a và b bằng các đường thẳng đứng màu xanh lá.
-

3. Thực hiện thuật toán tìm nghiệm

Tùy vào lựa chọn ở DropDown, chương trình chạy một trong ba logic sau:

A. Phương pháp Chia đôi (Bisection)

- **Điều kiện:** Kiểm tra $f(a) \cdot f(b) < 0$ (nghĩa là hàm số đổi dấu trong khoảng, đảm bảo có ít nhất một nghiệm).
- **Cách làm:** Liên tục chia đôi khoảng $[a, b]$. Nếu $f(\text{điểm giữa})$ cùng dấu với $f(a)$, nghiệm nằm ở nửa bên phải và ngược lại.
- **Dừng lại khi:** Chiều dài khoảng còn lại nhỏ hơn sai số ϵ .

B. Phương pháp Lặp đơn (Fixed-point Iteration)

- **Biến đổi:** Chuyển $f(x) = 0$ thành dạng $x = g(x)$. Trong mã, $g(x)$ được chọn tự động dựa trên đạo hàm: $g(x) = x - \frac{f(x)}{f'(x_0)}$.
- **Điều kiện hội tụ:** Kiểm tra $|g'(x)| < 1$ tại hai đầu mút. Nếu không thỏa mãn, thuật toán có thể không tìm thấy nghiệm.
- **Cách làm:** Tính $x_{k+1} = g(x_k)$.
- **Dừng lại khi:** Khoảng cách giữa hai lần lặp $|x_{curr} - x_{prev}| < \epsilon$.

C. Phương pháp Newton (Tiếp tuyến)

- **Cách làm:** Sử dụng tiếp tuyến của đồ thị tại điểm x_k để tìm điểm cắt trục hoành tiếp theo: $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$.
 - **Ưu điểm:** Tốc độ hội tụ rất nhanh.
 - **Dừng lại khi:** Hiệu số giữa hai lần lặp liên tiếp đạt mức sai số cho phép.
-

4. Hiển thị kết quả

- **Kết quả nghiệm:** Giá trị x tìm được sẽ được ghi vào ô "Kết quả nghiệm".
- **Số lần lặp:** Hiển thị tổng số bước tính toán mà máy đã thực hiện.
- **Trực quan hóa:** Một dấu chấm tròn màu xanh lá sẽ xuất hiện ngay trên đồ thị tại vị trí nghiệm vừa tìm được.
- **Thông báo lỗi:** Nếu không tìm thấy nghiệm (sau 100 lần lặp) hoặc dữ liệu nhập sai, một hộp thoại (UI Alert) sẽ xuất hiện để cảnh báo.

PHẦN 2: GIAO DIỆN NỘI SUY

1/ Code:

```
% Button pushed function: RunButton_2
function RunButton_2Pushed(app, event)

% Nội suy

% 1. THU THẬP VÀ XỬ LÝ DỮ LIỆU ĐẦU VÀO

try
    % str2num: chuyển chuỗi số cách nhau bằng dấu cách hoặc dấu phẩy thành vector
    x_data = str2num(app.NhpdliuxEditField.Value);
    y_data = str2num(app.NhpdliuyEditField.Value);
    x_interp = app.NhpgitrcnnisuyEditField.Value; % Giá trị cần nội suy
    phuong_phap = app.ChnphngphpnisuyDropDown.Value;

    if isempty(x_data) || isempty(y_data) || length(x_data) ~= length(y_data)
        uiaalert(app.UIFigure, 'Vui lòng nhập dữ liệu x và y hợp lệ (cùng kích thước).', 'Lỗi Dữ liệu');
        return;
    end
    n = length(x_data);
catch ME
    uiaalert(app.UIFigure, ['Lỗi nhập liệu: ' ME.message], 'Lỗi Đầu vào');
    return;
end

% 2. KHỞI TẠO BIẾN KẾT QUẢ VÀ SYMBOLIC

syms x;
P_sym = 0;
```

```
% 3. THỰC HIỆN TÍNH TOÁN NỘI SUY

if strcmp(phuong_phap, 'Lagrange') % PHƯƠNG PHÁP NỘI SUY LAGRANGE
    for i = 1:n
        L_i = 1;
        % Tính toán đa thức cơ sở Lagrange L_i(x)
        for j = 1:n
            if i ~= j
                L_i = L_i * (x - x_data(j)) / (x_data(i) - x_data(j));
            end
        end
        P_sym = P_sym + y_data(i) * L_i; % Cộng dồn đa thức P(x) = sum(y_i * L_i(x))
    end
elseif strcmp(phuong_phap, 'Newton') % PHƯƠNG PHÁP NỘI SUY NEWTON

    % Tính bảng sai phân chia
    F = zeros(n, n);
    F(:, 1) = y_data';

    for j = 2:n
        for i = j:n
            % Công thức sai phân chia: F[i, j] = (F[i, j-1] - F[i-1, j-1]) / (x_data[i] - x_data[i-j+1])
            F(i, j) = (F(i, j-1) - F(i-1, j-1)) / (x_data(i) - x_data(i-j+1));
        end
    end

    % Các hệ số Newton là các phần tử trên đường chéo chính của F: F(1, 1), F(2, 2), ..., F(n, n)
    coeffs = diag(F)';

    % Xây dựng đa thức nội suy Newton P(x)
    P_sym = coeffs(1); % Bắt đầu với f[x0]
    T = 1; % Tích số (x - x_i)
```

```

% Xây dựng đa thức nội suy Newton P(x)
P_sym = coeffs(1); % Bắt đầu với f[x0]
T = 1; % Tích số (x - x_i)

for k = 2:n
    % Tích T = (x - x_0)(x - x_1)...(x - x_{k-2})
    T = T * (x - x_data(k-1));
    P_sym = P_sym + coeffs(k) * T;
end
else
    uialert(app.UIFigure, 'Phương pháp nội suy chưa được chọn hoặc không hợp lệ.', 'Lỗi Thuật toán');
    return;
end

```

% 4. XỬ LÝ VÀ HIỂN THỊ KẾT QUẢ

```

% Rút gọn đa thức và làm đẹp về dạng thập phân
P_sym_simplified = vpa(simplify(P_sym), 6);

% Chuyển đa thức về dạng hàm để tính giá trị số
P_func = matlabFunction(P_sym);

% Tính kết quả nội suy số tại x_interp
y_interp = P_func(x_interp);

% Hiển thị Đa thức
app.KtquathcnisuyTextArea.Value = char(P_sym_simplified);

% Hiển thị Kết quả số
app.KtqunisuyEditField.Value = y_interp;

```

% 5. VẼ ĐỒ THỊ

```

try
    % Tạo khoảng X để vẽ đồ thị
    x_min = min(x_data) - 0.1 * (max(x_data) - min(x_data));
    x_max = max(x_data) + 0.1 * (max(x_data) - min(x_data));
    X_plot = linspace(x_min, x_max, 500);
    Y_plot = P_func(X_plot);

    % Xóa đồ thị cũ và thiết lập lại
    cla(app.UIAxes2);

    % Vẽ Đa thức nội suy
    plot(app.UIAxes2, X_plot, Y_plot, 'b-', 'LineWidth', 1.5, 'DisplayName', 'Đa thức nội suy');
    hold(app.UIAxes2, 'on');

    % Vẽ Dữ liệu thực
    plot(app.UIAxes2, x_data, y_data, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r', 'DisplayName', 'Dữ liệu thực');

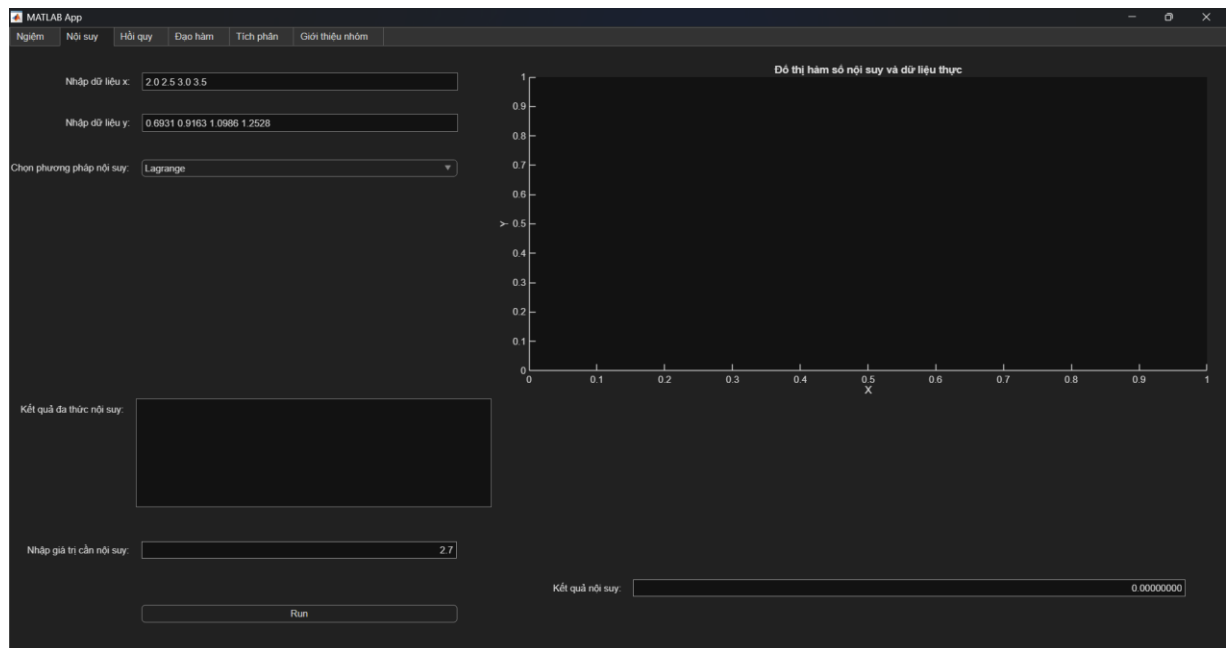
    % Đánh dấu giá trị cần nội suy
    plot(app.UIAxes2, x_interp, y_interp, 'gx', 'MarkerSize', 10, 'LineWidth', 2, 'DisplayName', 'Giá trị nội suy');

    % Thiết lập tiêu đề và nhãn
    title(app.UIAxes2, ['Nội suy ' phương_phap ' P(x)']);
    xlabel(app.UIAxes2, 'X');
    ylabel(app.UIAxes2, 'Y');
    legend(app.UIAxes2, 'show', 'Location', 'best');
    grid(app.UIAxes2, 'on');
    hold(app.UIAxes2, 'off');

catch ME_plot
    uialert(app.UIFigure, ['Lỗi khi vẽ đồ thị: ' ME_plot.message], 'Lỗi Đồ thị');
end

```

2/ Kết quả giao diện:



3/ Minh họa thuật toán:

1. Minh họa thuật toán nội suy (tổng quát)

Bước 1. Cho tập dữ liệu:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

Bước 2. Xây dựng đa thức nội suy $P(x)$ sao cho:

$$P(x_i) = y_i, i = 1, \dots, n$$

Bước 3. Tính giá trị xấp xỉ:

$$y \approx P(x^*)$$

2. Minh họa thuật toán Nội suy Lagrange

Ý tưởng: Xây dựng đa thức nội suy bằng các đa thức cơ sở.

Các bước:

1. Với mỗi điểm dữ liệu, xây dựng đa thức cơ sở:

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

2. Đa thức nội suy:

$$P(x) = \sum_{i=1}^n y_i L_i(x)$$

3. Giá trị nội suy tại x^* :

$$y^* = P(x^*)$$

3. Minh họa thuật toán Nội suy Newton

Ý tưởng: Xây dựng đa thức nội suy dựa trên sai phân chia.

Các bước:

1. Lập bảng sai phân chia:

$$f[x_i] = y_i$$

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

2. Xây dựng đa thức Newton:

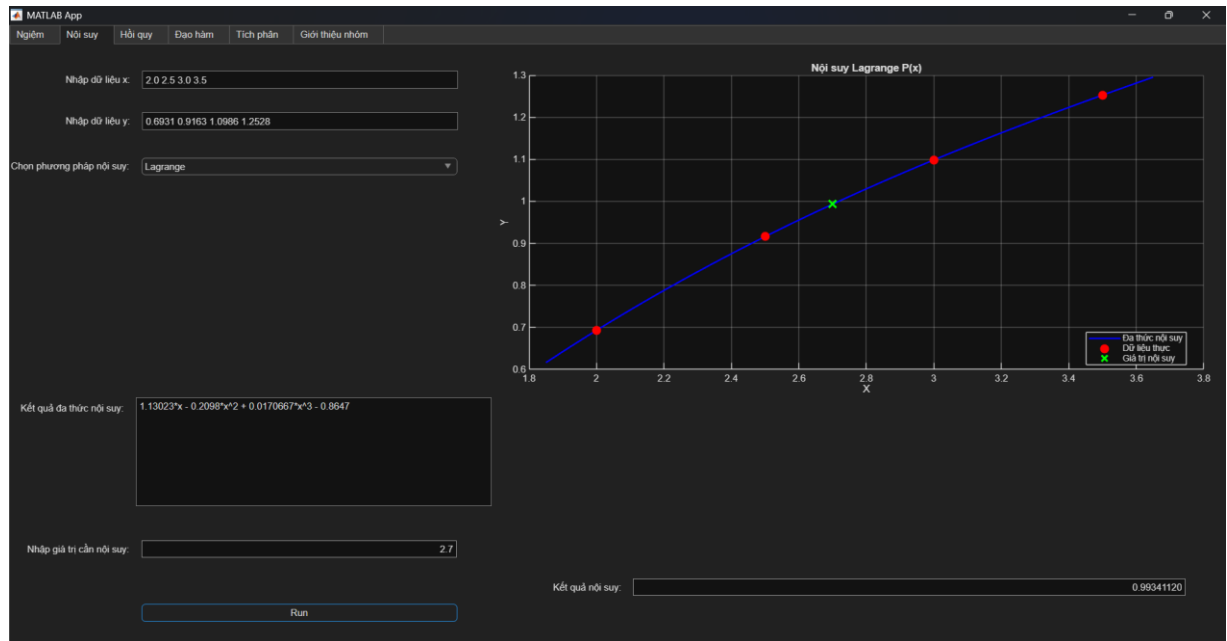
$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

3. Giá trị nội suy tại x^* :

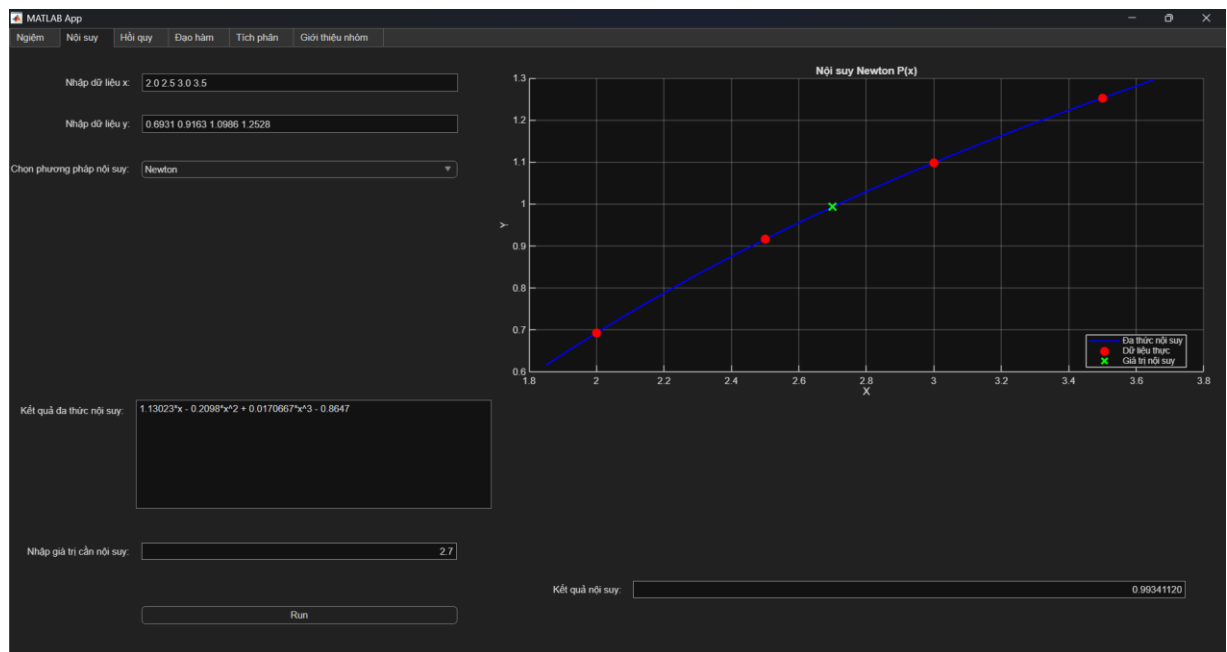
$$y^* = P(x^*)$$

4/ Minh họa kết quả:

- Phương pháp Lagrange:



- Phương pháp Newton:



5/ Mô tả hoạt động:

Mã nguồn là hàm xử lý cho tab Nội suy trong ứng dụng MATLAB. Mục đích của chương trình là xây dựng một đa thức đi qua các điểm dữ liệu cho trước và dự đoán giá trị tại một điểm mới.

Dưới đây là mô tả chi tiết hoạt động của chương trình:

1. Thu thập và kiểm tra dữ liệu đầu vào

- **Xử lý chuỗi:** Sử dụng lệnh `str2num` để chuyển danh sách các con số người dùng nhập (cách nhau bởi dấu cách hoặc phẩy) thành các vector tọa độ `x_data` và `y_data`.
 - **Kiểm tra tính logic:** Chương trình kiểm tra xem người dùng đã nhập đủ dữ liệu chưa và đảm bảo số lượng phần tử của `x` và `y` phải bằng nhau.
 - **Tham số khác:** Lấy giá trị `x` cần tính nội suy và phương pháp được chọn từ giao diện.
-

2. Thực hiện các thuật toán nội suy

Chương trình sử dụng thư viện Symbolic Math (`syms x`) để xây dựng biểu thức toán học tường minh cho đa thức.

Phương pháp Lagrange:

- **Nguyên lý:** Xây dựng các đa thức cơ sở $L_i(x)$ sao cho tại mỗi điểm x_i , đa thức đó bằng 1 và tại các điểm x_j khác thì bằng 0.
- **Kết quả:** Đa thức tổng quát là $P(x) = \sum y_i L_i(x)$.
-

Phương pháp Newton: (Sai phân chia)

- **Bảng sai phân:** Chương trình lập một ma trận vuông để tính toán các tỷ hiệu phân (divided differences).
 - **Xây dựng đa thức:** Sử dụng các hệ số nằm trên đường chéo của bảng sai phân để viết đa thức dưới dạng: $P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots$
-

3. Xử lý biểu thức và tính toán số

- **Làm đẹp đa thức:** Sử dụng lệnh `simplify` và `vpa` để rút gọn biểu thức toán học và hiển thị dưới dạng số thập phân (độ chính xác 6 chữ số) để người dùng dễ đọc.
 - **Tính giá trị cụ thể:** Chuyển biểu thức ký hiệu thành một hàm số (`matlabFunction`) để tính toán giá trị y tương ứng với điểm x nội suy mà người dùng đã nhập.
-

4. Hiển thị kết quả và Đồ thị

- **Kết quả văn bản:** Hiển thị công thức đa thức hoàn chỉnh vào ô "Kết quả đa thức" và giá trị số cụ thể vào ô "Kết quả nội suy".
- **Trực quan hóa đồ thị:**
 - Vẽ đường cong của đa thức nội suy bằng màu xanh lam (b-).
 - Vẽ các điểm dữ liệu gốc bằng các hình tròn đỏ (ro).
 - Đánh dấu điểm vừa nội suy bằng dấu "x" màu xanh lá (gx) để người dùng so sánh độ khớp.

PHẦN 3: GIAO DIỆN HỒI QUY

1/ Code:

```
% Button pushed function: RunButton_3
function RunButton_3Pushed(app, event)

% Hồi quy

% 1. THU THẬP VÀ XỬ LÝ DỮ LIỆU ĐẦU VÀO

try
    % Lấy dữ liệu X và Y
    x_data = str2num(app.NhpdliuxEditField_2.Value);
    y_data = str2num(app.NhpdliuyEditField_2.Value);

    % Lấy giá trị cần dự đoán
    x_predict = app.NhpgitrcndonEditField.Value;

    phuong_phap = app.ChnphngphpnisuyDropDown_2.Value;

    if isempty(x_data) || isempty(y_data) || length(x_data) ~= length(y_data)
        uialert(app.UIFigure, 'Vui lòng nhập dữ liệu x và y hợp lệ (cùng kích thước).', 'Lỗi Dữ liệu');
        return;
    end
    n = length(x_data);
catch ME
    uialert(app.UIFigure, ['Lỗi nhập liệu: ' ME.message], 'Lỗi Đầu vào');
    return;
end

% Khởi tạo các biến
syms x;
P_sym = 0;
reg_str = ''; % Chuỗi phương trình hồi quy
y_predict = NaN;

% Tính các tổng cần thiết cho Hồi quy Tuyến tính cơ bản: Y = a0 + a1*X
sum_x = sum(x_data);
sum_y = sum(y_data);
sum_x2 = sum(x_data.^2);
sum_xy = sum(x_data .* y_data);

% Ma trận hệ số [n sum(x); sum(x) sum(x^2)]
D = n * sum_x2 - sum_x^2;

if D == 0
    uialert(app.UIFigure, 'Dữ liệu không đủ hoặc trùng lặp, không thể thực hiện hồi quy tuyến tính.', 'Lỗi Thuật toán');
    return;
end

% 2. THỰC HIỆN TÍNH TOÁN HỒI QUY

if strcmp(phuong_phap, 'tuyến tính') % HỒI QUY TUYẾN TÍNH: Y = a0 + a1*X

    a1 = (n * sum_xy - sum_x * sum_y) / D;
    a0 = (sum_y * sum_x2 - sum_x * sum_xy) / D;

    P_sym = a0 + a1 * x;
    reg_str = sprintf('Y = %.4f + %.4f * X', a0, a1);

elseif strcmp(phuong_phap, 'hàm mũ') % HỒI QUY HÀM MŨ: Y = a*e^(b*X) (Tuyến tính hóa: ln(Y) = ln(a) + b*X)

    % Kiểm tra và xử lý logarit của Y
    if any(y_data <= 0)
        uialert(app.UIFigure, 'Giá trị Y phải > 0 để thực hiện Hồi quy Hàm Mũ.', 'Lỗi Dữ liệu');
        return;
    end
    Y_star = log(y_data); % Y* = ln(Y)
```

```

% Tính các tổng mới cho mô hình  $Y^* = A0 + A1 \cdot X$ 
sum_Y_star = sum(Y_star);
sum_X_Y_star = sum(x_data .* Y_star);

% Tính A1 (b) và A0 (ln(a))
A1 = (n * sum_X_Y_star - sum_x * sum_Y_star) / D; % A1 = b
A0 = (sum_Y_star * sum_x2 - sum_x * sum_X_Y_star) / D; % A0 = ln(a)

b = A1;
a = exp(A0);

P_sym = a * exp(b * x);
reg_str = sprintf('Y = %.4f * e^(%.4f * X)', a, b);

elseif strcmp(phuong_phap, 'Logarit') % HỒI QUY LOGARIT:  $Y = a0 + a1 \cdot \ln(X)$  (Tuyến tính hóa:  $Y = a0 + a1 \cdot X^*$ )

% Kiểm tra và xử lý logarit của X
if any(x_data <= 0)
    uialert(app.UIFigure, 'Giá trị X phải > 0 để thực hiện Hồi quy Logarit.', 'Lỗi Dữ liệu');
    return;
end
X_star = log(x_data); %  $X^* = \ln(X)$ 

% Tính các tổng mới cho mô hình  $Y = a0 + a1 \cdot X^*$ 
sum_X_star = sum(X_star);
sum_X_star2 = sum(X_star.^2);
sum_Y_X_star = sum(y_data .* X_star);

% Ma trận hệ số mới [n sum(X*); sum(X*) sum(X*^2)]
D_star = n * sum_X_star2 - sum_X_star^2;

if D_star == 0
    uialert(app.UIFigure, 'Dữ liệu ln(X) không đủ hoặc trùng lặp.', 'Lỗi Thuật toán');
    return;
end

```

```

% Tính a1 và a0
a1 = (n * sum_Y_X_star - sum_X_star * sum_y) / D_star;
a0 = (sum_y * sum_X_star2 - sum_X_star * sum_Y_X_star) / D_star;

P_sym = a0 + a1 * log(x);
reg_str = sprintf('Y = %.4f + %.4f * ln(X)', a0, a1);

end

% 3. TÍNH KẾT QUẢ DỰ ĐOÁN

if ~isnan(P_sym)
    % Chuyển đa thức về dạng hàm MATLAB để tính giá trị số
    P_func = matlabFunction(P_sym);

    % Kiểm tra điều kiện đầu vào cho hàm mũ/logarit khi dự đoán
    if strcmp(phuong_phap, 'Logarit') && x_predict <= 0
        uialert(app.UIFigure, 'Giá trị dự đoán X phải > 0 cho Hồi quy Logarit.', 'Lỗi Dự đoán');
        y_predict = NaN;
    else
        y_predict = P_func(x_predict);
    end
end

% 4. HIỂN THỊ KẾT QUẢ

app.KtquphngtrnhhiquyTextArea.Value = reg_str;
app.KtqudonEditField.Value = y_predict;

```

```

% 5. VẼ ĐỒ THỊ

try
    % Tạo khoảng X để vẽ đồ thị
    x_min = min(x_data);
    x_max = max(x_data);

    % Nếu hồi quy logarit và có giá trị dự đoán cần vẽ
    if strcmp(phuong_phap, 'Logarit') && x_predict > x_max
        x_max = x_predict;
    elseif strcmp(phuong_phap, 'Logarit') && x_predict < x_min
        x_min = x_predict;
    end

    % Điều chỉnh cho đồ thị đẹp hơn
    plot_min = x_min - 0.1 * (x_max - x_min);
    plot_max = x_max + 0.1 * (x_max - x_min);

    % Đảm bảo x > 0 cho logarit
    if strcmp(phuong_phap, 'Logarit') && plot_min <= 0
        plot_min = 0.001; % Đặt cận dưới > 0
    end

    X_plot = linspace(plot_min, plot_max, 500);
    Y_plot = P_func(X_plot);

    % Xóa đồ thị cũ và thiết lập lại
    cla(app.UIAxes2_2);

    % Vẽ Hàm Hồi quy (Đường cong)
    plot(app.UIAxes2_2, X_plot, Y_plot, 'b-', 'LineWidth', 2, 'DisplayName', 'Hàm Hồi quy');
    hold(app.UIAxes2_2, 'on');

    % Vẽ Dữ liệu thực
    scatter(app.UIAxes2_2, x_data, y_data, 80, 'ro', 'filled', 'DisplayName', 'Dữ liệu thực');

```

```

    % Vẽ Dữ liệu thực
    scatter(app.UIAxes2_2, x_data, y_data, 80, 'ro', 'filled', 'DisplayName', 'Dữ liệu thực');

    % Đánh dấu giá trị dự đoán
    if ~isnan(y_predict)
        scatter(app.UIAxes2_2, x_predict, y_predict, 100, 'gx', 'LineWidth', 2, 'DisplayName', 'Giá trị dự đoán');
    end

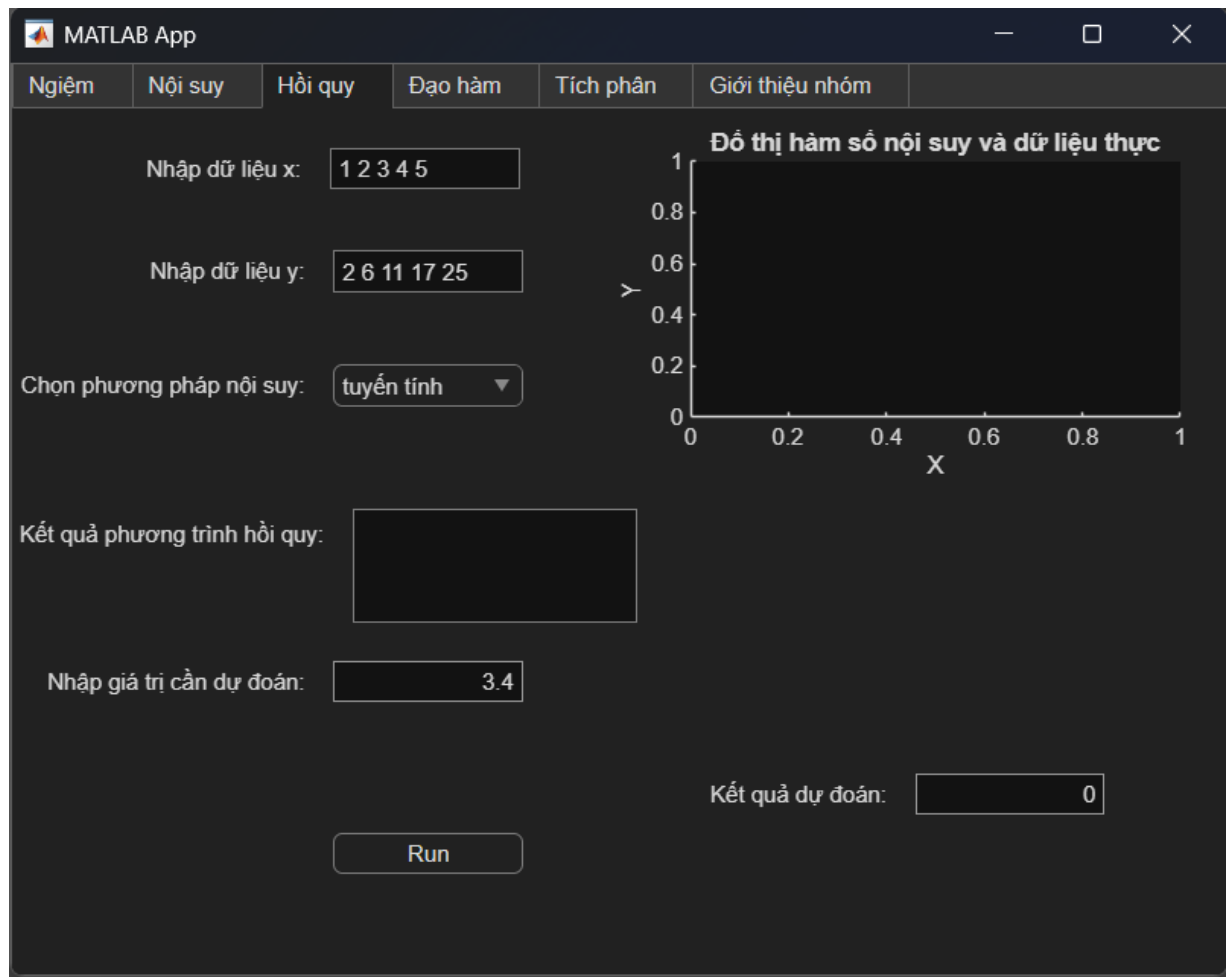
    % Thiết lập tiêu đề và nhãn
    title(app.UIAxes2_2, ['Hồi quy ' phuong_phap]);
    xlabel(app.UIAxes2_2, 'X');
    ylabel(app.UIAxes2_2, 'Y');
    legend(app.UIAxes2_2, 'show', 'Location', 'best');
    grid(app.UIAxes2_2, 'on');
    hold(app.UIAxes2_2, 'off');

catch ME_plot
    uialert(app.UIFigure, ['Lỗi khi vẽ đồ thị: ' ME_plot.message], 'Lỗi Đồ thị');
end

end

```

2/ Kết quả giao diện:



3/ Minh họa thuật toán:

1. Minh họa thuật toán hồi quy (tổng quát)

Bước 1. Cho tập dữ liệu thực nghiệm:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

Bước 2. Chọn dạng hàm hồi quy phù hợp.

Bước 3. Xác định các hệ số của mô hình theo **nguyên lý bình phương tối thiểu**.

Bước 4. Dùng mô hình thu được để dự đoán giá trị tại x^* :

$$y^* = f(x^*)$$

2. Minh họa thuật toán Hồi quy tuyến tính

Mô hình:

$$y = a_0 + a_1x$$

Các bước:

1. Tính các tổng:

$$\sum x_i, \sum y_i, \sum x_i^2, \sum x_i y_i$$

2. Giải hệ chuẩn để tìm hệ số:

$$a_1 = \frac{n \sum x_i y_i - (\sum x_i)(\sum y_i)}{n \sum x_i^2 - (\sum x_i)^2}$$

$$a_0 = \frac{\sum y_i \sum x_i^2 - (\sum x_i)(\sum x_i y_i)}{n \sum x_i^2 - (\sum x_i)^2}$$

3. Hàm hồi quy thu được:

$$y = a_0 + a_1x$$

4. Dự đoán:

$$y^* = a_0 + a_1x^*$$

3. Minh họa thuật toán Hồi quy hàm mũ

Mô hình:

$$y = ae^{bx}$$

Tuyến tính hóa:

$$\ln y = \ln a + bx$$

Các bước:

1. Đặt:

$$Y^* = \ln y$$

2. Thực hiện hồi quy tuyến tính cho cặp dữ liệu:

$$(x_i, Y_i^*)$$

3. Tìm được:

$$b, a = e^{\ln a}$$

4. Hàm hồi quy:

$$y = ae^{bx}$$

5. Dự đoán:

$$y^* = ae^{bx^*}$$

4. Minh họa thuật toán Hồi quy logarit

Mô hình:

$$y = a_0 + a_1 \ln x$$

Tuyến tính hóa:

$$X^* = \ln x$$

Các bước:

1. Thực hiện hồi quy tuyến tính cho dữ liệu:

$$(X_i^*, y_i)$$

2. Tìm được các hệ số:

$$a_0, a_1$$

3. Hàm hồi quy:

$$y = a_0 + a_1 \ln x$$

4. Dự đoán:

$$y^{*} = a_0 + a_1 \ln x^{*}$$

4/ Minh họa kết quả:

- Phương pháp tuyến tính:



- Phương pháp hàm mũ:



- Phương pháp Logarit:



5/ Mô tả hoạt động:

Mã nguồn là hàm xử lý cho tab **Hồi quy** (Regression) trong ứng dụng MATLAB. Mục đích của chương trình là tìm một quy luật hàm số (tuyến tính hoặc phi tuyến) khớp nhất với tập dữ liệu cho trước để thực hiện dự đoán các giá trị mới.

Dưới đây là mô tả chi tiết hoạt động của chương trình:

1. Thu thập và xử lý dữ liệu

- **Nhập liệu:** Chương trình lấy vector X và Y từ giao diện người dùng bằng lệnh str2num. Nếu độ dài hai vector không bằng nhau, ứng dụng sẽ báo lỗi.
- **Tham số dự đoán:** Người dùng nhập một giá trị x cụ thể để máy tính dự báo giá trị y tương ứng dựa trên mô hình tìm được.

2. Các mô hình hồi quy hỗ trợ

Dựa trên lựa chọn của người dùng, chương trình thực hiện tính toán theo một trong ba phương pháp:

A. Hồi quy Tuyến tính ($Y = a_0 + a_1X$)

- Sử dụng phương pháp bình phương tối thiểu để giải hệ phương trình chuẩn.
- Tính toán các tổng $\sum x$, $\sum y$, $\sum x^2$, $\sum xy$ để tìm hệ số chặn (a_0) và hệ số góc (a_1).

B. Hồi quy Hàm mũ ($Y = a \cdot e^{bX}$)

- **Tuyến tính hóa:** Chương trình thực hiện lấy logarit tự nhiên hai vế: $\ln(Y) = \ln(a) + bX$.
- **Điều kiện:** Toàn bộ giá trị Y phải dương ($Y > 0$).
- Sau khi tìm được hệ số trong không gian logarit, chương trình dùng hàm exp() để chuyển ngược lại về dạng hàm mũ gốc.

C. Hồi quy Logarit ($Y = a_0 + a_1 \cdot \ln(X)$)

- **Tuyến tính hóa:** Đặt $X^* = \ln(X)$ để đưa về dạng tuyến tính $Y = a_0 + a_1X^*$.
- **Điều kiện:** Toàn bộ giá trị X phải dương ($X > 0$).

3. Tính toán kết quả và hiển thị

- **Phương trình hồi quy:** Chuỗi phương trình (ví dụ: $Y = 1.234 + 0.567 * X$) được hiển thị trong vùng văn bản để người dùng biết quy luật tìm được.
- **Dự đoán:** Sử dụng hàm số vừa tìm được để tính giá trị $y_{predict}$ tại điểm $x_{predict}$ người dùng yêu cầu.

4. Trực quan hóa trên đồ thị

- **Đường hồi quy:** Vẽ đường biểu diễn quy luật bằng màu xanh lam (b-).
- **Dữ liệu thực tế:** Các điểm dữ liệu gốc được vẽ bằng các chấm tròn đỏ có tô màu (ro, filled) để người dùng so sánh độ khớp của mô hình.
- **Điểm dự đoán:** Giá trị cần dự báo được đánh dấu bằng dấu "x" màu xanh lá (gx) trên đồ thị.

PHẦN 4: GIAO DIỆN ĐẠO HÀM

1/ Code:

```
% Button pushed function: RunButton_4
function RunButton_4Pushed(app, event)

% Đạo hàm

% 1. THU THẬP DỮ LIỆU

phuong_phap = app.ChnphngphphmDropDown.Value;
sai_so = app.ChngitrsaisDropDown.Value;
x0 = app.NhpgitrcntnhohmEditField.Value;
derivative_result = NaN;

% Đọc dữ liệu đầu vào
x_data_discrete = str2num(app.NhpdliuxEditField_3.Value);
y_data_discrete = str2num(app.NhpdliuyEditField_3.Value);

f_str = strtrim(app.Nhphm_sEditField.Value); % Xóa khoảng trắng thừa
h = app.Nhphbc_hEditField.Value;

% Xác định chế độ nhập liệu
is_discrete_input = ~isempty(x_data_discrete) && ~isempty(y_data_discrete);
is_function_input = ~isempty(f_str) && ~isnan(h) && (h > 0);

if is_discrete_input && is_function_input
    uialert(app.UIFigure, 'Vui lòng chỉ nhập MỘT trong hai bộ dữ liệu.', 'Lỗi Đầu vào');
    return;
elseif ~is_discrete_input && ~is_function_input
    uialert(app.UIFigure, 'Vui lòng nhập Dữ liệu rời rạc HOẶC Hàm số/Bước h.', 'Lỗi Đầu vào');
    return;
end

% 2. XỬ LÝ THEO TRƯỜNG HỢP

% TH1: Nhập dữ liệu
if is_discrete_input

% TH1: Nhập dữ liệu
if is_discrete_input
    x_data = x_data_discrete;
    y_data = y_data_discrete;
    if length(x_data) ~= length(y_data)
        uialert(app.UIFigure, 'Dữ liệu x và y phải cùng kích thước.', 'Lỗi Dữ liệu'); return;
    end

% Tìm chỉ số i và gán giá trị f_i
[val, i] = min(abs(x_data - x0));
if val > 1e-6 % Kiểm tra nếu x0 không thực sự nằm trong dữ liệu
    uialert(app.UIFigure, 'Giá trị x0 phải nằm trong tập dữ liệu x đã nhập.', 'Lỗi x0'); return;
end

f_i = y_data(i);
n = length(x_data);
h_discrete = x_data(2) - x_data(1);

try
    if strcmp(phuong_phap, 'Xấp xỉ tiến') % NEWTON TIEN
        if strcmp(sai_so, '0(h)')
            if i + 1 > n, error('L'); end
            derivative_result = (y_data(i+1) - f_i) / h_discrete;
        else % 0(h^2)
            if i + 2 > n, error('L'); end
            derivative_result = (-y_data(i+2) + 4*y_data(i+1) - 3*f_i) / (2*h_discrete);
        end
    elseif strcmp(phuong_phap, 'Xấp xỉ lùi') % NEWTON LUI
        if strcmp(sai_so, '0(h)')
            if i - 1 < 1, error('L'); end
            derivative_result = (f_i - y_data(i-1)) / h_discrete;
        else % 0(h^2)
            if i - 2 < 1, error('L'); end
            derivative_result = (3*f_i - 4*y_data(i-1) + y_data(i-2)) / (2*h_discrete);
        end
    end
catch
    derivative_result = NaN;
end

end
```

```

elseif strcmp(phuong_phap, 'Xấp xỉ lùi') % NEWTON LUI
    if strcmp(sai_so, '0(h)')
        if i - 1 < 1, error('L'); end
        derivative_result = (f_i - y_data(i-1)) / h_discrete;
    else % 0(h^2)
        if i - 2 < 1, error('L'); end
        derivative_result = (3*f_i - 4*y_data(i-1) + y_data(i-2)) / (2*h_discrete);
    end

elseif strcmp(phuong_phap, 'Xấp xỉ trung tâm') % NEWTON TRUNG TAM
    if i - 1 < 1 || i + 1 > n, error('L'); end
    derivative_result = (y_data(i+1) - y_data(i-1)) / (2*h_discrete);
end
catch
    uialert(app.UIFigure, 'Không đủ điểm dữ liệu lân cận để tính.', 'Lỗi Chỉ số'); return;
end

% TH2: Nhập hàm
elseif is_function_input
    try
        % Xử lý chuỗi để MATLAB hiểu arcsin là asin
        f_str_fixed = strrep(f_str, 'arcsin', 'asin');
        f_str_fixed = strrep(f_str_fixed, 'arccos', 'acos');
        f_str_fixed = strrep(f_str_fixed, 'arctan', 'atan');

        syms x
        f_sym = str2sym(f_str_fixed);
        f_func = matlabFunction(f_sym);

        % Tính toán các điểm cần thiết
        f_x0 = f_func(x0);
        if strcmp(phuong_phap, 'Xấp xỉ tiến') % NEWTON TIEN
            if strcmp(sai_so, '0(h)')
                derivative_result = (f_func(x0 + h) - f_x0) / h;
            else

```

```

                % Tính toán các điểm cần thiết
                f_x0 = f_func(x0);
            if strcmp(phuong_phap, 'Xấp xỉ tiến') % NEWTON TIEN
                if strcmp(sai_so, '0(h)')
                    derivative_result = (f_func(x0 + h) - f_x0) / h;
                else
                    derivative_result = (-f_func(x0 + 2*h) + 4*f_func(x0 + h) - 3*f_x0) / (2*h);
                end

            elseif strcmp(phuong_phap, 'Xấp xỉ lùi') % NEWTON LUI
                if strcmp(sai_so, '0(h)')
                    derivative_result = (f_x0 - f_func(x0 - h)) / h;
                else
                    derivative_result = (3*f_x0 - 4*f_func(x0 - h) + f_func(x0 - 2*h)) / (2*h);
                end

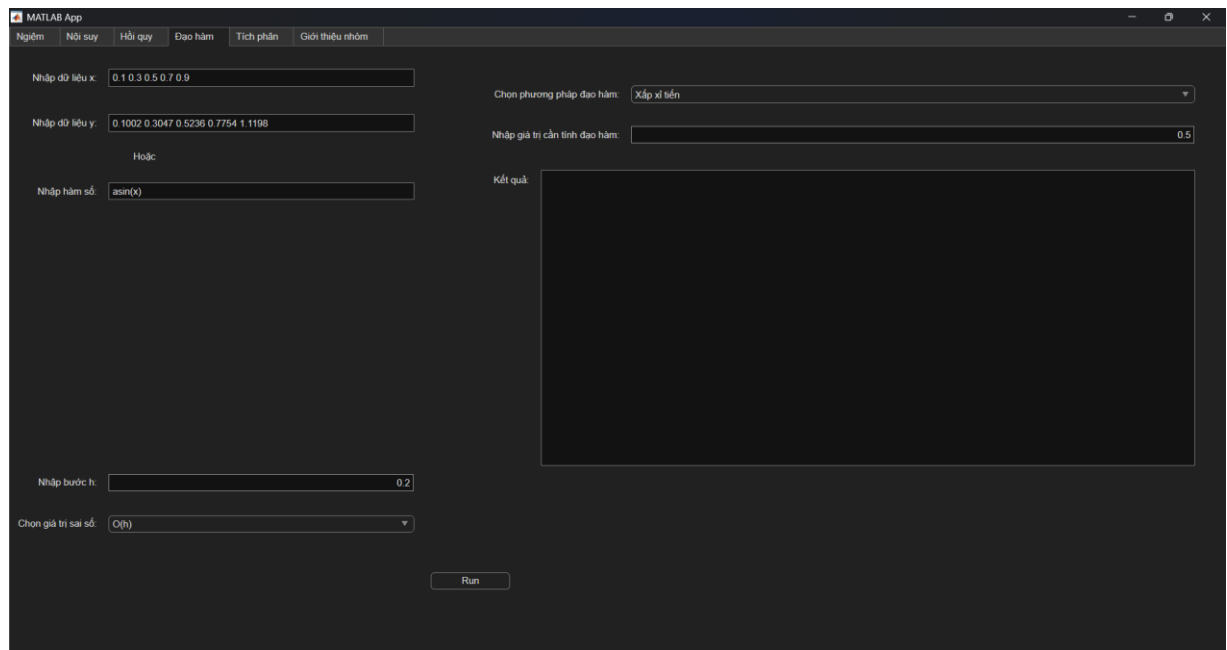
            elseif strcmp(phuong_phap, 'Xấp xỉ trung tâm') % NEWTON TRUNG TAM
                derivative_result = (f_func(x0 + h) - f_func(x0 - h)) / (2*h);
            end
        catch
            uialert(app.UIFigure, 'Lỗi hàm số hoặc bước h không hợp lệ.', 'Lỗi'); return;
        end
    end

    % 3. HIỂN THỊ
    app.KtquTextArea.Value = sprintf('%0.8f', derivative_result);

end
end

```


2/ Kết quả giao diện:



3/ Minh họa thuật toán:

1. Minh họa thuật toán đạo hàm số (tổng quát)

Mục tiêu: Xấp xỉ đạo hàm cấp 1 của hàm số tại điểm x_0 .

Dữ liệu: Hàm $f(x)$ hoặc bảng giá trị rời rạc, bước $h > 0$.

Điều kiện dừng: Sai số bậc $O(h)$ hoặc $O(h^2)$.

2. Minh họa thuật toán xấp xỉ tiến

Công thức bậc $O(h)$:

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}$$

Công thức bậc $O(h^2)$:

$$f'(x_0) \approx \frac{-f(x_0 + 2h) + 4f(x_0 + h) - 3f(x_0)}{2h}$$

3. Minh họa thuật toán xấp xỉ lùi

Công thức bậc $O(h)$:

$$f'(x_0) \approx \frac{f(x_0) - f(x_0 - h)}{h}$$

Công thức bậc $O(h^2)$:

$$f'(x_0) \approx \frac{3f(x_0) - 4f(x_0 - h) + f(x_0 - 2h)}{2h}$$

4. Minh họa thuật toán xấp xỉ trung tâm

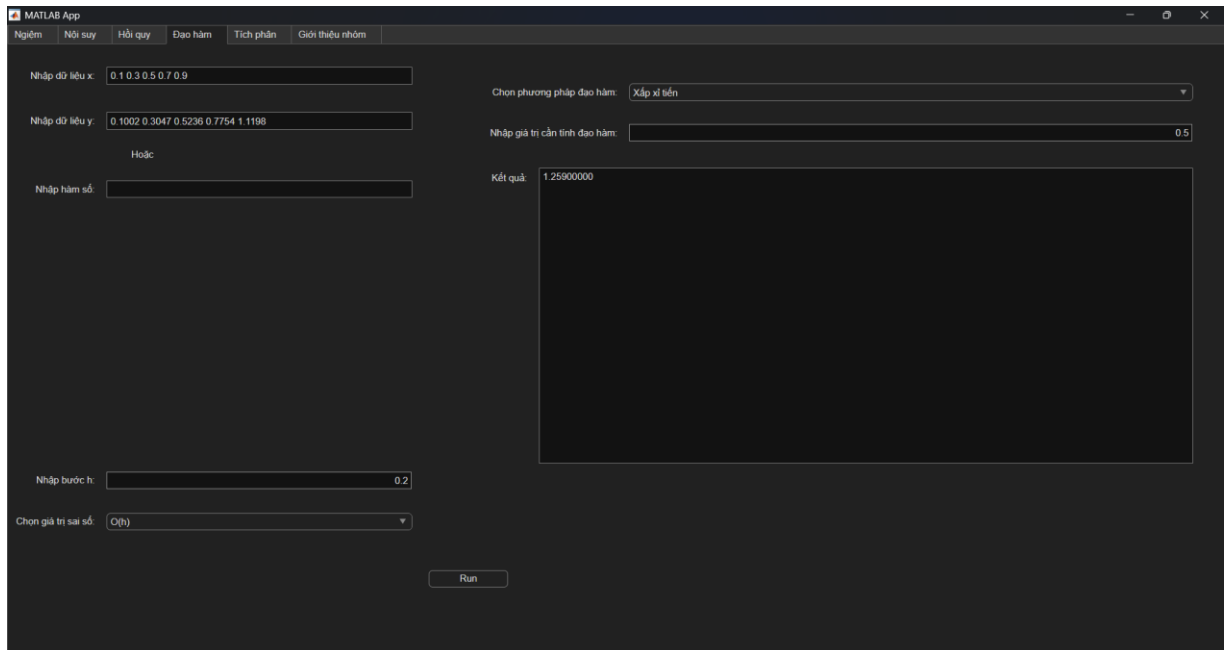
Công thức bậc $O(h^2)$:

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}$$

4/ Minh họa kết quả:

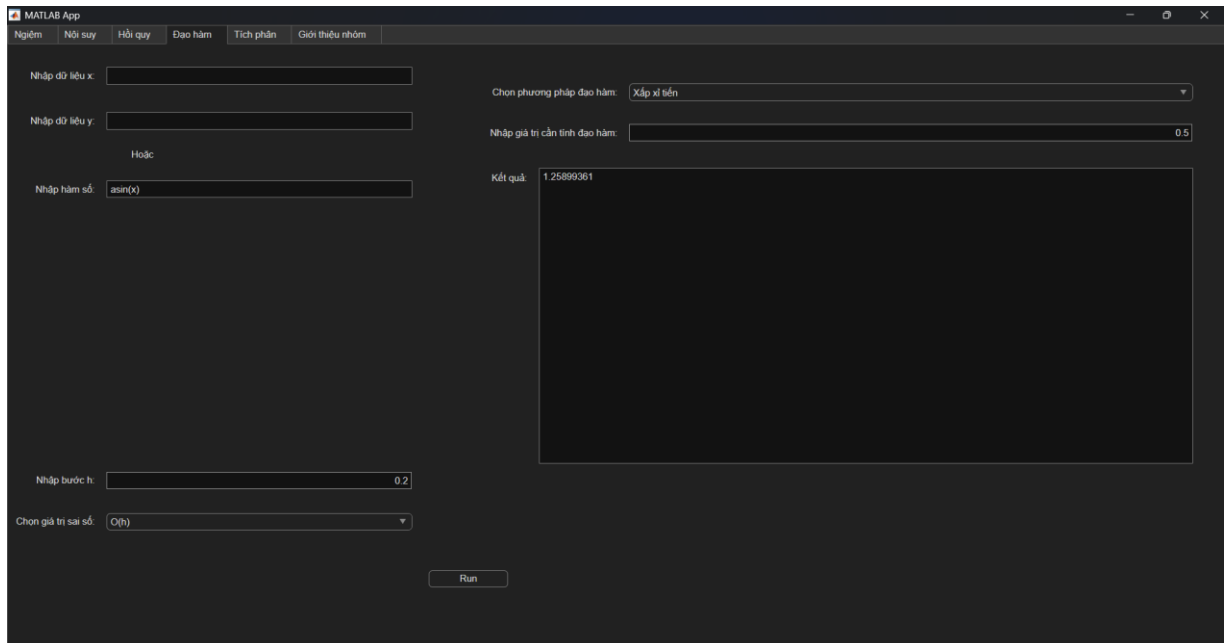
Giá trị sai $O(h)$:

Phương pháp đạo hàm xấp xỉ tiến



The MATLAB App interface displays the following data:

Input	Value
Nhập dữ liệu x:	0.1 0.3 0.5 0.7 0.9
Nhập dữ liệu y:	0.1002 0.3047 0.5236 0.7754 1.1198
Nhập hàm số:	
Nhập bước h:	0.2
Chọn giá trị sai số:	$O(h)$
Chọn phương pháp đạo hàm:	Xấp xỉ tiến
Nhập giá trị cần tính đạo hàm:	0.5
Kết quả:	1.25900000



The MATLAB App interface displays the following data:

Input	Value
Nhập dữ liệu x:	
Nhập dữ liệu y:	
Nhập hàm số:	asin(x)
Nhập bước h:	0.2
Chọn giá trị sai số:	$O(h)$
Chọn phương pháp đạo hàm:	Xấp xỉ tiến
Nhập giá trị cần tính đạo hàm:	0.5
Kết quả:	1.25899361

Phương pháp đạo hàm xấp xỉ lùi

MATLAB App

Ngõem Nội suy Hồi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x: 0.1 0.3 0.5 0.7 0.9

Nhập dữ liệu y: 0.1002 0.3047 0.5236 0.7754 1.1198

Hoặc

Nhập hàm số:

Nhập bước h: 0.2

Chọn giá trị sai số: O(h)

Chon phương pháp đạo hàm: Xấp xỉ lùi

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.09450000

Run

MATLAB App

Ngõem Nội suy Hồi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x:

Nhập dữ liệu y:

Hoặc

Nhập hàm số: asin(x)

Nhập bước h: 0.2

Chọn giá trị sai số: O(h)

Chon phương pháp đạo hàm: Xấp xỉ lùi

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.09453061

Run

Phương pháp đạo hàm xấp xỉ trung tâm

MATLAB App

Thử nghiệm Nội suy Hồi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x: 0.1 0.3 0.5 0.7 0.9

Nhập dữ liệu y: 0.1002 0.3047 0.5236 0.7754 1.1198

Hoặc

Nhập hàm số:

Nhập bước h: 0.2

Chọn giá trị sai số: 0(h)

Chon phương pháp đạo hàm: Xấp xỉ trung tâm

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.17675000

Run

MATLAB App

Thử nghiệm Nội suy Hồi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x:

Nhập dữ liệu y:

Hoặc

Nhập hàm số: asin(x)

Nhập bước h: 0.2

Chọn giá trị sai số: 0(h)

Chon phương pháp đạo hàm: Xấp xỉ trung tâm

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.17676211

Run

Giá trị sai $O(h^2)$:

Phương pháp đạo hàm xấp xỉ tiến

The MATLAB App interface shows the following settings and results:

- Navigation tabs:** Ngõm, Nội suy, Hồi quy, Đạo hàm, Tích phân, Giới thiệu nhóm.
- Input fields:**
 - Nhập dữ liệu x: 0 1 0.3 0.5 0.7 0.9
 - Nhập dữ liệu y: 0.1002 0.3047 0.5236 0.7754 1.1198
 - Hoặc
 - Nhập hàm số: (empty)
- Method selection:** Chọn phương pháp đạo hàm: Xấp xỉ tiến
- Step size:** Nhập giá trị cần tính đạo hàm: 0.5
- Result:** Kết quả: 1.02750000
- Step size input:** Nhập bước h: 0.2
- Error order selection:** Chọn giá trị sai số: $O(h^2)$
- Run button:** Run

The MATLAB App interface shows the following settings and results:

- Navigation tabs:** Ngõm, Nội suy, Hồi quy, Đạo hàm, Tích phân, Giới thiệu nhóm.
- Input fields:**
 - Nhập dữ liệu x: (empty)
 - Nhập dữ liệu y: (empty)
 - Hoặc
 - Nhập hàm số: $\sin(x)$
- Method selection:** Chọn phương pháp đạo hàm: Xấp xỉ tiến
- Step size:** Nhập giá trị cần tính đạo hàm: 0.5
- Result:** Kết quả: 1.02756036
- Step size input:** Nhập bước h: 0.2
- Error order selection:** Chọn giá trị sai số: $O(h^2)$
- Run button:** Run

Phương pháp đạo hàm xấp xỉ lùi

MATLAB App

Ngắm Nội suy Hồi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x: 0.1 0.3 0.5 0.7 0.9

Nhập dữ liệu y: 0.1002 0.3047 0.5236 0.7754 1.1198

Hoặc

Nhập hàm số:

Nhập bước h: 0.2

Chọn giá trị sai số: $O(h^2)$

Chon phương pháp đạo hàm: Xấp xỉ lùi

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.13050000

Run

MATLAB App

Ngắm Nội suy Hồi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x:

Nhập dữ liệu y:

Hoặc

Nhập hàm số: $\sin(x)$

Nhập bước h: 0.2

Chọn giá trị sai số: $O(h^2)$

Chon phương pháp đạo hàm: Xấp xỉ lùi

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.13048283

Run

Phương pháp đạo hàm xấp xỉ trung tâm

MATLAB App

Ngôn ngữ | Nội suy | Hồi quy | Đạo hàm | Tích phân | Giới thiệu nhóm

Nhập dữ liệu x: 0.1 0.3 0.5 0.7 0.9

Nhập dữ liệu y: 0.1002 0.3047 0.5236 0.7754 1.1198

Hoặc

Nhập hàm số:

Nhập bước h: 0.2

Chọn giá trị sai số: $O(h^2)$

Chon phương pháp đạo hàm: Xấp xỉ trung tâm

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.17675000

Run

MATLAB App

Ngôn ngữ | Nội suy | Hồi quy | Đạo hàm | Tích phân | Giới thiệu nhóm

Nhập dữ liệu x:

Nhập dữ liệu y:

Hoặc

Nhập hàm số: $\sin(x)$

Nhập bước h: 0.2

Chọn giá trị sai số: $O(h^2)$

Chon phương pháp đạo hàm: Xấp xỉ trung tâm

Nhập giá trị cần tính đạo hàm: 0.5

Kết quả: 1.17676211

Run

5/ Mô tả hoạt động:

Mã nguồn thuộc về tab **Đạo hàm** trong ứng dụng MATLAB App Designer. Chương trình này cho phép tính toán đạo hàm tại một điểm bằng phương pháp xấp xỉ số học dựa trên hai loại dữ liệu đầu vào: dữ liệu rời rạc (bảng giá trị) hoặc hàm số toán học. Dưới đây là mô tả chi tiết các bước hoạt động của chương trình:

1. Thu thập và Phân loại dữ liệu đầu vào

Chương trình bắt đầu bằng việc kiểm tra người dùng nhập dữ liệu theo cách nào:

- **Cách 1: Dữ liệu rời rạc:** Nhập tập hợp các điểm x và y tương ứng.
 - **Cách 2: Hàm số:** Nhập biểu thức toán học (ví dụ: $\sin(x)$), điểm cần tính x_0 , và bước nhảy h .
 - **Ràng buộc:** Ứng dụng sẽ báo lỗi nếu người dùng nhập cả hai loại dữ liệu cùng lúc hoặc không nhập gì cả.
-

2. Các phương pháp xấp xỉ đạo hàm

Tùy thuộc vào lựa chọn tại menu "Chọn phương pháp đạo hàm", mã nguồn thực hiện các công thức toán học khác nhau:

A. Xấp xỉ tiến (Forward Difference)

Sử dụng các điểm phía sau điểm x_0 để tính độ dốc:

- **Sai số $O(h)$:** $f'(x_0) \approx \frac{f(x_0+h)-f(x_0)}{h}$.
- **Sai số $O(h^2)$:** Sử dụng thêm điểm $(x_0 + 2h)$ để tăng độ chính xác.

B. Xấp xỉ lùi (Backward Difference)

Sử dụng các điểm phía trước điểm x_0 :

- **Sai số $O(h)$:** $f'(x_0) \approx \frac{f(x_0)-f(x_0-h)}{h}$.
- **Sai số $O(h^2)$:** Sử dụng thêm điểm $(x_0 - 2h)$.

C. Xấp xỉ trung tâm (Central Difference)

Sử dụng điểm trước và sau của x_0 để tính đạo hàm (thường có độ chính xác cao hơn):

- **Công thức:** $f'(x_0) \approx \frac{f(x_0+h)-f(x_0-h)}{2h}$.
-

3. Quy trình xử lý kỹ thuật

- **Đối với dữ liệu rời rạc:** Chương trình tự động tìm chỉ số của điểm x_0 trong mảng và tính khoảng cách h giữa các điểm dữ liệu. Nếu không đủ điểm lân cận (ví dụ tính xấp xỉ tiến tại điểm cuối cùng của mảng), ứng dụng sẽ báo lỗi "Không đủ điểm dữ liệu".
 - **Đối với hàm số:** Mã nguồn sử dụng thư viện Symbolic (syms x) để hiểu các hàm lượng giác và chuyển đổi chúng thành hàm MATLAB có thể tính toán được (matlabFunction).
-

4. Hiển thị kết quả

- Kết quả cuối cùng được làm tròn và hiển thị trong ô **Kết quả** với độ chính xác 8 chữ số thập phân.
- Nếu có bất kỳ lỗi nào trong quá trình nhập liệu hoặc tính toán (ví dụ: x_0 không nằm trong tập dữ liệu), một thông báo uialert sẽ xuất hiện để hướng dẫn người dùng.

PHẦN 5: GIAO DIỆN TÍCH PHÂN

1/ Code:

```
% Button pushed function: RunButton_5
function RunButton_5Pushed(app, event)

% Tích phân

% 1. THU THẬP VÀ XÁC ĐỊNH PHƯƠNG THỨC NHẬP DỮ LIỆU

phuong_phap = app.ChnpngphptchphnDropDown.Value;
I_result = NaN; % Tích phân kết quả

% ĐỌC DỮ LIỆU ĐẦU VÀO

% Bộ 1: Dữ liệu rời rạc (x, y)
x_data = str2num(app.NhpdliuxEditField_4.Value);
y_data = str2num(app.NhpdliuyEditField_4.Value);

% Bộ 2: Hàm số, Cận, và N
f_str = app.NhphmsEditField.Value;
N = app.NhpNEditField.Value;

% Xử lý chuỗi cận 'a, b'
can_str = app.NhpcntnhtchphnEditField.Value;
try
    can_parts = str2num(can_str);
    a = can_parts(1);
    b = can_parts(2);
catch
    a = NaN; b = NaN;
end

% Xác định chế độ
is_discrete_input = ~isempty(x_data) && ~isempty(y_data);
is_function_input = ~isempty(f_str) && ~isnan(a) && ~isnan(b) && (a ~= b) && (N > 0);

if is_discrete_input && is_function_input
```

```
    if is_discrete_input && is_function_input
        uialert(app.UIFigure, 'Vui lòng chỉ nhập MỘT trong hai bộ dữ liệu.', 'Lỗi Đầu vào');
        return;
    elseif ~is_discrete_input && ~is_function_input
        uialert(app.UIFigure, 'Vui lòng nhập Dữ liệu rời rạc HOẶC Hàm số/Cận/N hợp lệ.', 'Lỗi Đầu vào');
        return;
    end

% 2. XỬ LÝ THEO TRƯỜNG HỢP NHẬP LIỆU

% TRƯỜNG HỢP 1: TÍNH TỪ DỮ LIỆU RỜI RẠC (X, Y)
if is_discrete_input
    if length(x_data) ~= length(y_data)
        uialert(app.UIFigure, 'Dữ liệu x và y phải cùng kích thước.', 'Lỗi Dữ liệu');
        return;
    end
    n_points = length(x_data);
    N_sections = n_points - 1; % Số đoạn

    % Giả định khoảng cách h đều
    h = x_data(2) - x_data(1);

    % Kiểm tra tính hợp lệ của N
    if any(diff(diff(x_data)) > 1e-9) % Kiểm tra h có đều không
        uialert(app.UIFigure, 'Phương pháp tích phân yêu cầu khoảng cách x (h) phải đều.', 'Lỗi Dữ liệu');
        return;
    end

    % Tính toán tích phân
    try
        if strcmp(phuong_phap, 'Hình thang') % Phương pháp Hình thang
            I_result = h/2 * (y_data(1) + 2*sum(y_data(2:N_sections)) + y_data(n_points));
```

```

elseif strcmp(phuong_phap, 'Simpson 1/3') % Simpson 1/3
    if mod(N_sections, 2) ~= 0
        uialert(app.UIFigure, 'Simpson 1/3 yêu cầu số đoạn N phải chẵn (số điểm phải lẻ).', 'Lỗi Quy tắc');
        return;
    end

    I_result = h/3 * (y_data(1) + y_data(n_points) + 4*sum(y_data(2:2:n_points-1)) + 2*sum(y_data(3:2:n_points-2)));

elseif strcmp(phuong_phap, 'Simpson 3/8') % Simpson 3/8
    if mod(N_sections, 3) ~= 0
        uialert(app.UIFigure, 'Simpson 3/8 yêu cầu số đoạn N phải là bội số của 3.', 'Lỗi Quy tắc');
        return;
    end

    % Công thức Hợp thành Simpson 3/8 (phức tạp hơn, đây là cách tổng quát)
    I_result = 0;
    for k = 1:3:N_sections
        I_result = I_result + 3*h/8 * (y_data(k) + 3*y_data(k+1) + 3*y_data(k+2) + y_data(k+3));
    end
end
catch ME
    uialert(app.UIFigure, ['Lỗi tính toán tích phân rời rạc: ' ME.message], 'Lỗi Tính toán');
    return;
end

% TRƯỜNG HỢP 2: TÍNH TỬ HÀM SỐ (f(x))
elseif is_function_input
    try
        syms x
        f_sym = str2sym(f_str);
        f_func = matlabFunction(f_sym);
    catch ME

```

```

% TRƯỜNG HỢP 2: TÍNH TỬ HÀM SỐ (f(x))
elseif is_function_input
    try
        syms x
        f_sym = str2sym(f_str);
        f_func = matlabFunction(f_sym);
    catch ME
        uialert(app.UIFigure, 'Hàm số nhập vào không hợp lệ.', 'Lỗi Hàm số');
        return;
    end

    h = (b - a) / N;

    % Tạo các điểm x
    x_points = linspace(a, b, N + 1);
    y_points = f_func(x_points);

    try
        if strcmp(phuong_phap, 'Hình thang') % Phương pháp Hình thang
            I_result = h/2 * (y_points(1) + 2*sum(y_points(2:N)) + y_points(N+1));

            elseif strcmp(phuong_phap, 'Simpson 1/3') % Simpson 1/3
                if mod(N, 2) ~= 0
                    uialert(app.UIFigure, 'Simpson 1/3 yêu cầu số đoạn N phải chẵn.', 'Lỗi Quy tắc');
                    return;
                end

                I_result = h/3 * (y_points(1) + y_points(N+1) + 4*sum(y_points(2:2:N)) + 2*sum(y_points(3:2:N-1)));

            elseif strcmp(phuong_phap, 'Simpson 3/8') % Simpson 3/8
                if mod(N, 3) ~= 0
                    uialert(app.UIFigure, 'Simpson 3/8 yêu cầu số đoạn N phải là bội số của 3.', 'Lỗi Quy tắc');
                    return;
                end
            end
        end
    catch ME

```

```

elseif strcmp(phuong_phap, 'Simpson 3/8') % Simpson 3/8
    if mod(N, 3) ~= 0
        uialert(app UIFigure, 'Simpson 3/8 yêu cầu số đoạn N phải là bội số của 3.', 'Lỗi Quy tắc');
        return;
    end

    I_result = 0;
    for k = 1:3:N
        I_result = I_result + 3*h/8 * (y_points(k) + 3*y_points(k+1) + 3*y_points(k+2) + y_points(k+3));
    end
end
catch ME
    uialert(app UIFigure, ['Lỗi tính toán tích phân hàm số: ' ME.message], 'Lỗi Tính toán');
    return;
end
end

% 3. HIỂN THỊ KẾT QUẢ
if ~isnan(I_result)
    app.KtquEditField.Value = I_result;
else
    app.KtquEditField.Value = NaN;
    uialert(app UIFigure, 'Không thể tính tích phân. Kiểm tra lại dữ liệu và quy tắc N.', 'Lỗi Kết quả');
end

end

```

2/ Kết quả giao diện:

The screenshot shows the MATLAB App interface with the following components:

- Navigation Bar:** Includes tabs for 'Ngõm', 'Nôi suy', 'Hỏi quy', 'Đạo hàm', 'Tích phân', and 'Giới thiệu nhóm'. The 'Tích phân' (Integration) tab is currently selected.
- Input Fields:**
 - Nhập dữ liệu x:** A text box containing the values '0.2 0.4 0.6 0.8'.
 - Nhập dữ liệu y:** A text box containing the values '0.1823 0.3365 0.4700 0.5878'.
 - Nhập hàm số:** A text box containing the expression 'log(1 + x)'.
- Buttons:** A 'Hoặc' (Or) button is located between the x and y input fields.
- Method Selection:** A dropdown menu labeled 'Chọn phương pháp tích phân' (Choose integration method) is set to 'Hình thang' (Trapezoidal).
- Step Size:** A text box labeled 'Nhập cân để tính tích phân' (Enter step size for integration) contains the value '0.8'.
- Number of Segments:** A text box labeled 'Nhập N' (Enter N) contains the value '3'.
- Result Display:** A text box labeled 'Kết quả' (Result) shows the value '0'.
- Execution:** A 'Run' button is located at the bottom center of the interface.

3/ Minh họa thuật toán:

1. Minh họa thuật toán tích phân số (tổng quát)

Mục tiêu: Xấp xỉ tích phân xác định

$$I = \int_a^b f(x) dx$$

Dữ liệu: Hàm $f(x)$ hoặc bảng giá trị rời rạc, chia đoạn thành N khoảng đều.

Bước chia:

$$h = \frac{b - a}{N}$$

2. Minh họa thuật toán Hình thang

Ý tưởng: Xấp xỉ diện tích bằng các hình thang.

$$I \approx \frac{h}{2} \left[y_0 + 2 \sum_{i=1}^{N-1} y_i + y_N \right]$$

3. Minh họa thuật toán Simpson 1/3

Điều kiện: N chẵn.

$$I \approx \frac{h}{3} \left[y_0 + y_N + 4 \sum_{i \text{ lẻ}} y_i + 2 \sum_{i \text{ chẵn}} y_i \right]$$

4. Minh họa thuật toán Simpson 3/8

Điều kiện: N là bội của 3.

$$I \approx \frac{3h}{8} \left[y_0 + y_N + 3 \sum_{i \not\equiv 0 \pmod{3}} y_i + 2 \sum_{i \equiv 0 \pmod{3}} y_i \right]$$

4/ Minh họa kết quả:

Phương pháp hình thang:

The MATLAB App interface displays the 'Tích phân' (Integration) tab. The 'Chọn phương pháp tích phân' (Choose integration method) dropdown is set to 'Hình thang' (Trapezoidal). The input fields are filled with: 'Nhập dữ liệu x:' [0.0 0.2 0.4 0.6 0.8 1.0 1.2], 'Nhập dữ liệu y:' [0.0 0.1823 0.3365 0.4700 0.5878 0.6931 0.7885], and 'Nhập hàm số:' []. The 'Nhập cận để tính tích phân:' (Input limits for integration) field is set to [0 1.2], and 'Nhập N:' (Input N) is set to [6]. The 'Kết quả' (Result) field shows the value 0.5328. A 'Run' button is at the bottom.

The MATLAB App interface is the same as the previous one, but the 'Nhập hàm số:' (Input function) field is now filled with the expression $\log(1 + x)$. The 'Kết quả' (Result) field still shows 0.5328.

Phương pháp simpson 1/3:

The MATLAB App interface shows the 'Tích phân' (Integration) tab with the 'Chọn phương pháp tích phân' (Choose integration method) dropdown set to 'Simpson 1/3'. All other input fields and the 'Kết quả' (Result) field (0.5346) remain the same as in the previous screenshots.

MATLAB App

Ngệmm Nôi suy Hôi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x:

Nhập dữ liệu y:

Hoặc

Nhập hàm số:

Chọn phương pháp tích phân:

Nhập cận để tính tích phân:

Nhập N:

Kết quả:

Run

Phương pháp simpson 3/8:

MATLAB App

Ngệmm Nôi suy Hôi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x:

Nhập dữ liệu y:

Hoặc

Nhập hàm số:

Chọn phương pháp tích phân:

Nhập cận để tính tích phân:

Nhập N:

Kết quả:

Run

MATLAB App

Ngệmm Nôi suy Hôi quy Đạo hàm Tích phân Giới thiệu nhóm

Nhập dữ liệu x:

Nhập dữ liệu y:

Hoặc

Nhập hàm số:

Chọn phương pháp tích phân:

Nhập cận để tính tích phân:

Nhập N:

Kết quả:

Run

5/ Mô tả hoạt động:

Mã nguồn thuộc về tab **Tích phân** (Numerical Integration) trong ứng dụng MATLAB. Chương trình này cho phép tính diện tích dưới đường cong bằng các phương pháp xấp xỉ số học nổi tiếng.

Dưới đây là mô tả chi tiết hoạt động của chương trình:

1. Thu thập và xác định chế độ nhập liệu

Chương trình bắt đầu bằng việc kiểm tra người dùng nhập dữ liệu theo cách nào trong hai cách sau:

- **Cách 1: Dữ liệu rời rạc (x, y):** Người dùng nhập danh sách các điểm tọa độ. Chương trình kiểm tra khoảng cách giữa các điểm x (bước h) phải đều nhau.
 - **Cách 2: Hàm số toán học (f(x)):** Người dùng nhập biểu thức hàm (ví dụ: $\log(1 + x)$), các cận [a, b], và số đoạn chia N.
 - **Ràng buộc:** Bạn chỉ được chọn một trong hai cách. Nếu nhập cả hai hoặc không nhập gì, ứng dụng sẽ báo lỗi.
-

2. Các phương pháp tính tích phân hỗ trợ

Tùy vào lựa chọn trong Menu, chương trình áp dụng các công thức xấp xỉ khác nhau:

A. Phương pháp Hình thang (Trapezoidal Rule)

- **Nguyên lý:** Xấp xỉ diện tích dưới đường cong bằng các hình thang nhỏ.
- **Công thức:** $I \approx \frac{h}{2} * [y_0 + 2 \sum_{i=1}^{n-1} y_i + y_n]$.

B. Phương pháp Simpson 1/3

- **Nguyên lý:** Sử dụng các đường Parabol để xấp xỉ đường cong, cho độ chính xác cao hơn hình thang.
- **Điều kiện bắt buộc:** Số đoạn chia N phải là **số chẵn**.
- **Công thức:** $I \approx \frac{h}{3} * [y_0 + y_n + 4 \sum_{i=1}^{n-1} y_i + 2 \sum_{i=2}^{n-2} y_i]$. (4 – i chẵn; 2 – i lẻ)

C. Phương pháp Simpson 3/8

- **Nguyên lý:** Sử dụng đa thức bậc ba để xấp xỉ.
 - **Điều kiện bắt buộc:** Số đoạn chia N phải là **bội số của 3**.
 - **Công thức:** $I \approx \sum \frac{3h}{8} * [y_k + 3y_{k+1} + 3y_{k+2} + y_{k+3}]$.
-

3. Xử lý lỗi và Kiểm tra quy tắc

Chương trình rất chú trọng vào việc kiểm tra tính hợp lệ để tránh sai số:

- **Kiểm tra N:** Nếu chọn Simpson 1/3 mà N lẻ, hoặc Simpson 3/8 mà N không chia hết cho 3, ứng dụng sẽ hiện cảnh báo uialert ngay lập tức.
 - **Xử lý Symbolic:** Khi nhập hàm số dưới dạng văn bản, chương trình dùng str2sym để hiểu các ký hiệu toán học và matlabFunction để biến chúng thành hàm có thể tính toán số.
-

4. Hiển thị kết quả

- Kết quả tích phân cuối cùng sẽ được ghi vào ô **Kết quả** trên giao diện.
- Nếu có lỗi trong quá trình tính toán (như hàm số không hợp lệ), kết quả sẽ hiển thị NaN (Not a Number).

PHẦN 6: GIỚI THIỆU NHÓM

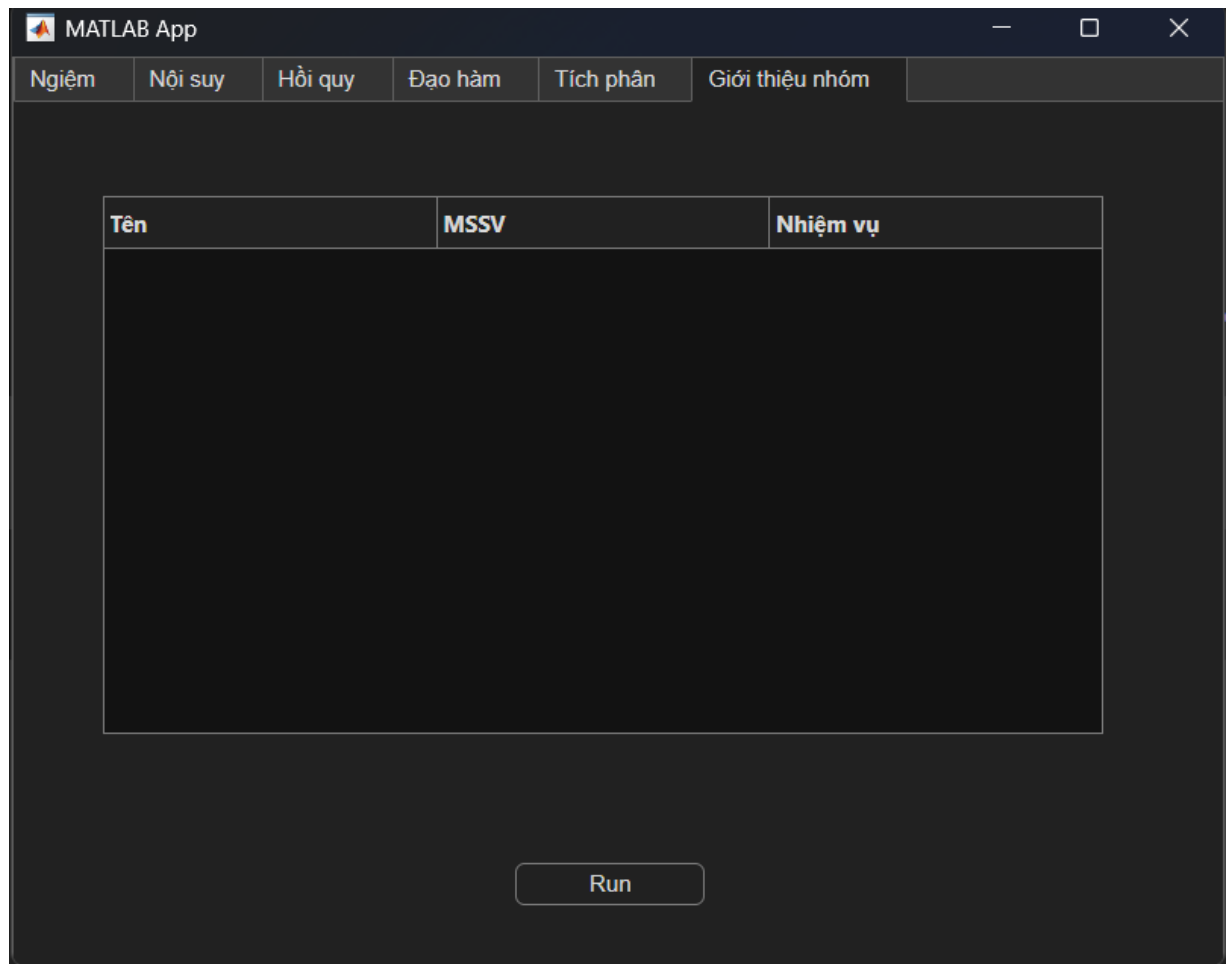
1/ Code:

```
% Button pushed function: RunButton_6
function RunButton_6Pushed(app, event)

    t = readtable("Book1.xlsx", "Sheet", 1)
    app.UITable.Data = t;

end
```

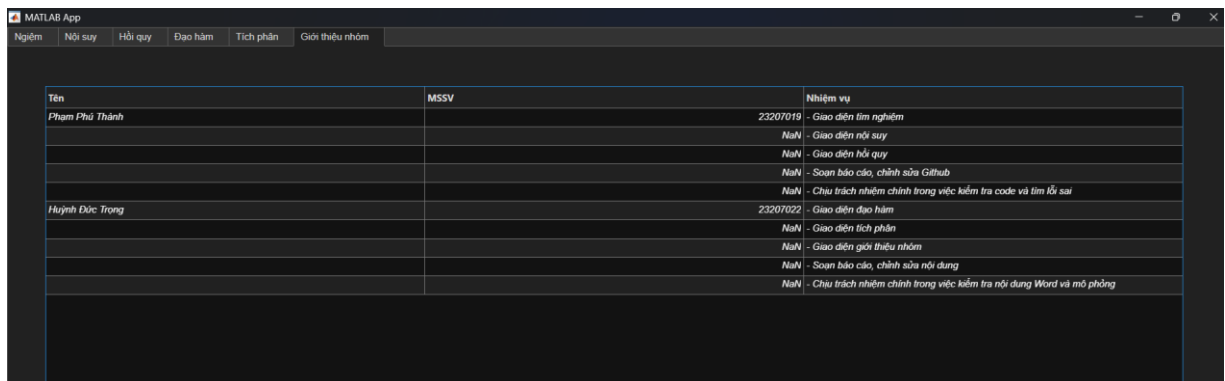
2/ Kết quả giao diện:



3/ Minh họa thuật toán:

- Mở file Excel
- Đọc dữ liệu từ Sheet 1
- Lưu dữ liệu vào biến bảng
- Hiển thị dữ liệu lên UITableView

4/ Minh họa kết quả:



The screenshot shows the MATLAB App interface with the 'Giới thiệu nhóm' (Group Introduction) tab selected. It displays a table with three columns: 'Tên' (Name), 'MSSV' (Student ID), and 'Nhiệm vụ' (Task). The table contains data for two students: Phạm Phú Thành and Huỳnh Đức Trọng. The tasks listed include 'Giao diện tìm nghiệm', 'Giao diện nội suy', 'Giao diện hồi quy', 'Soạn báo cáo, chỉnh sửa Github', and 'Chịu trách nhiệm chính trong việc kiểm tra code và tìm lỗi sai'.

Tên	MSSV	Nhiệm vụ
Phạm Phú Thành	23207019	- Giao diện tìm nghiệm
	NaN	- Giao diện nội suy
	NaN	- Giao diện hồi quy
	NaN	- Soạn báo cáo, chỉnh sửa Github
	NaN	- Chịu trách nhiệm chính trong việc kiểm tra code và tìm lỗi sai
Huỳnh Đức Trọng	23207022	- Giao diện đạo hàm
	NaN	- Giao diện tích phân
	NaN	- Giao diện giới thiệu nhóm
	NaN	- Soạn báo cáo, chỉnh sửa nội dung
	NaN	- Chịu trách nhiệm chính trong việc kiểm tra nội dung Word và mô phỏng

5/ Mô tả hoạt động:

Mã nguồn thuộc về tab Giới thiệu nhóm (Group Introduction) trong ứng dụng MATLAB. Tab này đóng vai trò là nơi quản lý thông tin nhân sự thực hiện dự án và tích hợp tính năng đọc dữ liệu từ tệp tin bên ngoài.

- **Hiển thị danh sách thành viên:**
 - Sử dụng một bảng dữ liệu (**UITableView**) để liệt kê chi tiết các thành viên thực hiện dự án.
- **Chức năng nạp dữ liệu từ Excel:**
 - Phía dưới bảng thông tin có một nút bấm **Run**.
 - Khi người dùng nhấn nút này, chương trình sẽ thực hiện lệnh `readtable("Book1.xlsx", "Sheet", 1)`.
 - Lệnh này sẽ tìm và đọc tệp Excel có tên Book1.xlsx tại trang tính (sheet) đầu tiên, sau đó gán toàn bộ dữ liệu này vào bảng `app.UITable.Data` để hiển thị lên giao diện.

TỔNG KẾT VÀ ĐÁNH GIÁ:

A. Tiêu chí đánh giá ứng dụng: (70%)

TT	Nội dung	Điểm	Ghi chú
1	Thiết kế được giao diện Tab Nghiệm	0.4	ĐẠT.
2	Thiết kế được giao diện Tab Nội Suy	0.4	ĐẠT.
3	Thiết kế được giao diện Tab Hồi quy	0.4	ĐẠT.
4	Thiết kế được giao diện Tab Đạo hàm	0.4	ĐẠT.
5	Thiết kế được giao diện Tab Tích phân	0.4	ĐẠT.
6	Thiết kế được giao diện Tab Giới thiệu nhóm	0.4	ĐẠT.
7	Tìm được nghiệm đúng phương pháp Chia đôi	0.4	ĐẠT.
8	Tìm được nghiệm đúng phương pháp Lặp	0.4	ĐẠT.
9	Tìm được nghiệm đúng phương pháp Newton	0.4	ĐẠT.
10	Vẽ được hàm số cần tìm nghiệm	0.4	ĐẠT.
11	Tìm được đa thức nội suy Newton	0.4	ĐẠT.
12	Dự đoán được giá trị cần nội suy với nội suy Newton	0.4	ĐẠT.
13	Tìm được đa thức nội suy Lagrange	0.4	ĐẠT.
14	Dự đoán được giá trị cần nội suy với nội suy Lagrange	0.4	ĐẠT.
15	Tìm được đa thức và vẽ phương trình hồi quy tuyến tính	0.4	ĐẠT.
16	Tìm được đa thức và vẽ phương trình hồi quy hàm mũ	0.4	ĐẠT.
17	Tìm được đa thức và vẽ phương trình hồi quy mũ e	0.4	ĐẠT.
18	Tính được đạo hàm cho dữ liệu x, y	0.4	ĐẠT.
19	Tính được đạo hàm từ hàm số	0.4	ĐẠT.
20	Thay đổi được phương pháp tính đạo hàm: xấp xỉ tiến, xấp xỉ lùi, xấp xỉ trung tâm	0.4	ĐẠT.
21	Tính được tích phân hình thang từ x, y	0.4	ĐẠT.
22	Tính được tích phân hình thang từ hàm số nhập vào	0.4	ĐẠT.
23	Tính được tích phân bằng phương pháp Simpson 1/3	0.4	ĐẠT.
24	Tính được tích phân bằng phương pháp Simpson 3/8	0.4	ĐẠT.
25	Có sử dụng hàm cho từng phương pháp	0.4	ĐẠT.

B. Thang điểm đánh giá báo cáo word: (30%)

TT	Nội dung	Điểm	Ghi chú
1	Có bìa, phần giới thiệu đề tài, danh sách thành viên	1.0	ĐẠT.
2	Có minh họa thuật toán cho từng phương pháp	2.5	ĐẠT. (Dựa vào lý thuyết hoặc những thuật toán các nhóm tự nghĩ ra)
3	Có mô tả hoạt động của ứng dụng	1.5	ĐẠT.
4	Có minh họa kết quả cho từng phương pháp - Chỉ ra test cho từng phương pháp	2.0	ĐẠT.
5	Chỉ ra những hạn chế hay những trường hợp nhóm chưa giải quyết được - Đề xuất hướng giải quyết nếu chưa thực hiện được trong đề tài	1.0	ĐẠT. (Chi tiết bên dưới)
6	Có sử dụng phần mềm quản lý code để cộng tác (Github)	1.0	ĐẠT.
7	Phản đánh giá nhóm - Phân công nhiệm vụ - Nội dung thực hiện từng cá nhân	1.0	ĐẠT. (Chi tiết bên dưới)
	- Có tự đánh giá điểm theo tiêu chí - Có tự đánh giá nhóm (Tổng các thành viên là 100%)		Điểm: 10. (Chi tiết bên dưới)

Nội dung	Trình bày
Hạn chế	- Nhóm vẫn còn phụ thuộc vào sự hỗ trợ của chatbot trong quá trình thực hiện đề tài. * Một số lỗi đã mắc trong quá trình: - Thử với những phương trình đòi hỏi số lần lặp cao (hơn 50 lần) gặp vấn đề. - Vòng lặp Newton/Lặp: Giới hạn 100 lần nhưng nếu không hội tụ, $x_nghiem = NaN$ mà không reset đồ thị. - Phân đạo hàm, các phương pháp ban đầu gặp vấn đề trùng lặp kết quả và $O(h^2)$: Chỉ số vượt ($i + 2 > n$ hoặc $i - 2 < 1$) .
Trường hợp chưa giải quyết triệt để	- Vòng lặp Newton/Lặp: Giới hạn 100 lần nhưng nếu không hội tụ, $x_nghiem = NaN$ mà không reset đồ thị. - Lỗi đạo hàm, vẫn còn tình trạng trùng lặp kết quả và $O(h^2)$: Chỉ số vượt ($i + 2 > n$ hoặc $i - 2 < 1$) .
Hướng giải quyết	- Tìm hiểu và đọc kỹ hơn tài liệu liên quan. Tuy nhiên, sai số vẫn còn xuất hiện nên cần thử nghiệm với nhiều phương pháp lặp khác nhau và cần thử nhiều lần. - Xác định được vấn đề nằm ở đâu, nhưng không giải quyết triệt để được.

Bảng đánh giá nhóm

Bảng đánh giá nhóm					
Họ tên	MSSV	Nội dung thực hiện	Trách nhiệm chính	Tự đánh giá (%)	
				Theo đồ án	Theo nhiệm vụ
Phạm Phú Thành	23207019	- Giao diện tìm nghiệm - Giao diện nội suy - Giao diện hồi quy - Soạn báo cáo, chỉnh sửa Github	Kiểm tra code và tìm lỗi sai	50%	100%
Huỳnh Đức Trọng	23207022	- Giao diện đạo hàm - Giao diện tích phân - Giao diện giới thiệu nhóm - Soạn báo cáo, chỉnh sửa nội dung	Kiểm tra nội dung Word và mô phỏng		
Đối chiếu lẫn nhau các nội dung được phân công, có so sánh thông qua giải trực tiếp (trên giấy) và các bài giải đã được kiểm chứng trong môn học				100%	
